# SW2PRJ2 Projekt

Energistyring og embeddede systemer til overvågning/fjernstyring

**Gruppe 2**

Afleveringsdato: 29/05-2026

Omfang: 71940

# Resumé / Abstract


## Resumé

Dette projekt omhandler udviklingen af et vand og energimonitoreringssystem til overvågning af vandforbrug af en brugers brusebadssessioner i private husstande. Formålet med systemet er at synliggøre brugerens vand og energiforbrug i realtid for derved at øge deres bevidsthed om deres ressourceforbrug og potentielt ændre deres adfærd. Systemet giver samtidig mulighed for, at flere brugere i samme husstand kan sammenligne deres forbrug gennem en rangliste, og derved motivere hinanden til et mindre ressourceforbrug.

Systemet består af flow og temperatursensorer til måling af vandmængde og temperatur samt en SOMO og højtaler, der giver brugeren auditiv feedback, ved vandmængde forbrug imens brugeren er i bad. Sensorerne kommunikerer via Arduinoer med en Raspberry Pi, som håndterer beregninger, dataopsamling og brugergrænsefladen. Brugeren interagerer med systemet gennem en touchskærm med tilhørende GUI, hvor live data, statistik og forbrug kan tilgås.

Projektet er dokumenteret med ASE modellen, hvor der er udarbejdet kravspecifikation, arkitektur, design samt integrationstest og accepttest af systemet. Systemet er udviklet som en funktionel prototype, der danner grundlag for videreudvikling af en løsning, som på sigt vil kunne implementeres i private husstande

## Abstract

This project is about the development of a system that can monitor water and energy consumption of individuals shower habits in a private household.

The purpose of the system is to give the individual user the opportunity to monitor their water and energy usage while in a shower, and to become aware of their resource usage impact, and potentially change their shower habits for the better.  The system also gives the users the opportunity to compare their shower habits with the other users through a ranking list and thereby motivating them to compete and lower their resource consumption.

The system consists of flow and temperature sensors for measuring water volume and water temperature, as well as sound module, with speaker, to notify the current user of their water consumption while using the shower. The sensors communicate through Arduinos that are connected to a Raspberry Pi, which keeps track of calculations, storing data and user interface. The User interacts with the system through a touchscreen and GUI, where live data and statistics can be accessed by the user.

The project is documented using the “ASE model”, where requirement specification, architecture, design, as well as an integration and acceptance test have been carried out to document the system. The system has been developed as prototype, testing the basic principles of the system that in the future could form the foundation for a fully implemented household solution.

# Forord

Denne projektrapport er udarbejdet af projektgruppe 2 på 2. semester ved Aarhus Universitet, og omhandler projektet *Smart Shower*.

Projektgruppen består af studerende fra softwareteknologiuddannelsen. Navnene på projektgruppens deltagere fremgår af rapportens forside.

Denne rapport understøttes af bilag, som beskriver udviklingsprocessen samt yderligere dokumentation af arbejdet. Derudover er alle benyttede datablad vedlagt som bilag.

Projektrapporten er afleveret den 29/05 2026.

## Ansvarsområder

Herunder beskrives ansvarsområder samt arbejdsfordelingen for udfærdigelse af projektet. Ansvarsområderne opdeles i primære og sekundære ansvarsområder. Primære ansvarsområder omfatter opgaver med hovedansvar for rettidig udførelse, mens sekundære ansvarsområder omfatter opgaver, hvor der ydes støtte eller assistance uden det overordnede ansvar.

Gruppen har i fællesskab udarbejdet kravsspecifikationen, accepttestspecifikationen samt systemarkitekturen.

# Indhold

Resumé / Abstract ..... 2
Forord ..... 3
Ansvarsområder ..... 4
1. Indledning ..... 7
1.1 Udfordringen ..... 7
1.2 Løsningskoncept ..... 7
1.3 Projektformulering ..... 8
2. Krav ..... 9
2.1 Aktør-kontekst diagram ..... 9
2.2 Use Cases ..... 9
2.3 Funktionelle krav ..... 10
2.4 Ikke-funktionelle krav ..... 11
2.5 Afgrænsning ..... 12
3. Metode og proces ..... 13
3.1 ASE-modellen ..... 13
3.2 SysML og UML ..... 14
4. Analyse ..... 15
4.1 Sensorer ..... 15
4.2 GUI ..... 16
5. Arkitektur ..... 18
5.1 Hardwarearkitektur ..... 18
5.2 Softwarearkitektur ..... 21
6. Design og implementering ..... 24
6.1 Hardware design ..... 24
6.2 Software design ..... 28
7. Integration og test ..... 47
7.1 Integrationstest ..... 47
8. Accepttest ..... 49
8.1 Prækondition for accepttest ..... 49
8.2 Accepttest funktionelle krav ..... 51
8.3 Accepttest non-funktionelle krav ..... 53

9. Diskussion ..... 56
10. Konklusion ..... 57
11. Fremtidigt arbejde ..... 58
Bilagsoversigt ..... 60
Referencer ..... 60

# 1. Indledning

## 1.1 Udfordringen

Mennesker forbruger meget energi, og der ses en stigende grad af ressourcespild, på en allerede belastet klode. Vand og energi er begrænsede ressourcer, og særligt forbruget af varmt brugsvand udgør en væsentlig del af både husholdningers vandforbrug og energiforbrug. Lange og varme bade medfører ikke blot et højt vandforbrug, men også et betydeligt energiforbrug til opvarmning af vandet, hvilket bidrager til øgede CO2-udledninger og højere forbrugeromkostninger. Samtidig mangler mange brugere indsigt i deres faktiske forbrug og de økonomiske og miljømæssige konsekvenser heraf. En kombination af måling, og løbende brugerfeedback kan bidrage til øget forbrugsbevidsthed, og dermed hjælpe brugeren i at reducere unødigt brug af ressourceforbruget.



*Figur 1: Global opvarmning*

## 1.2 Løsningskoncept

Formålet med dette projekt er at udvikle et system, der kan gøre vand- og energiforbrug mere synligt, forståeligt og sammenligneligt for brugerne. Vi har valgt at fokusere på brusebade, og vil med vores system *Smart Shower* give vores bruger konkret feedback på både pris og miljøpåvirkning af et bad. Systemet skal desuden understøtte ranglister, der motiverer til mere bæredygtig adfærd gennem sammenligning og forbrugsvisualisering. For yderligere at øge brugerens bevidsthed om det løbende vandforbrug afspilles der en lyd ved hver foruddefineret mængde vand, som fungerer som realtidsfeedback under badet.

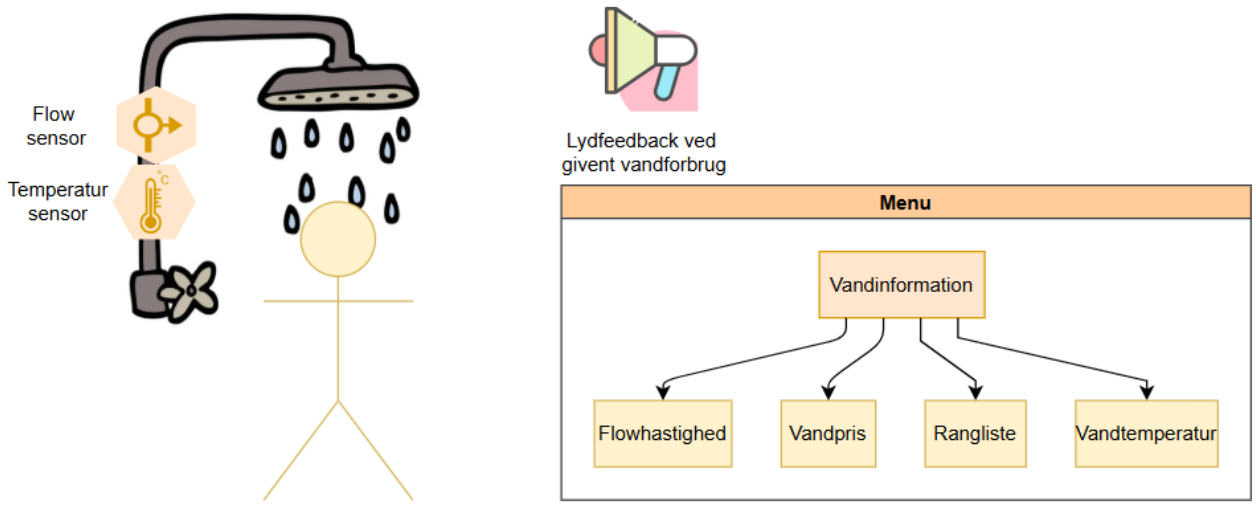
## 1.3 Projektformulering

Forbruget af varmt vand i husholdninger er højt og udgør en betydelig del af både vand- og energiforbruget. Det er almindeligt, at bruseren er tændt i længere tid end nødvendigt, uden at man tænker over, hvor mange liter vand og hvor meget energi der bliver brugt.

Opvarmning af vand kræver energi, hvilket både medfører øgede økonomiske omkostninger og øget $CO_2$-udledning. Problemet er, at forbruget ofte først bliver synligt, når regningen kommer, og der mangler løbende indsigt i det aktuelle forbrug.

Manglende bevidsthed om eget forbrug gør det svært for brugeren at ændre adfærd og reducere vand- og energispild.

Projektets problem er derfor:
Hvordan kan man øge brugerens bevidsthed om sit vandforbrug, så unødigt vand- og energiforbrug reduceres?

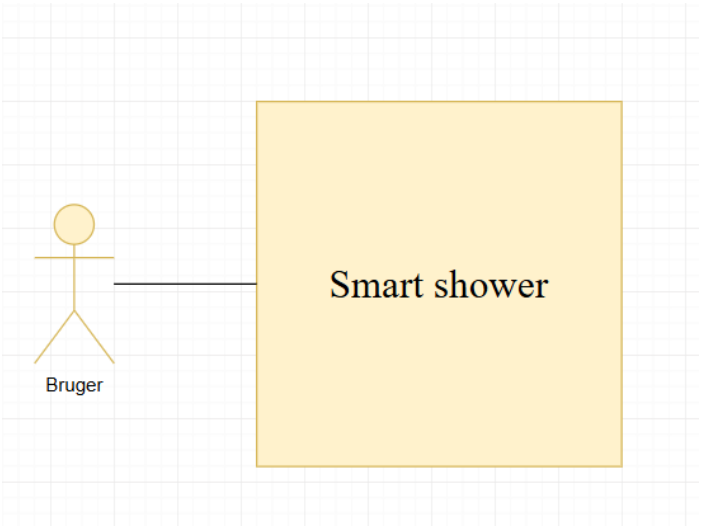



*Figur 2: Skitse over systemet*

# 2. Krav

Dette afsnit omhandler kravene der er specificeret for projektet. Afsnittet indeholder en kort beskrivelse af aktører, use cases samt funktionelle og ikke-funktionelle krav. For en mere detaljeret beskrivelse henvises til bilag A.

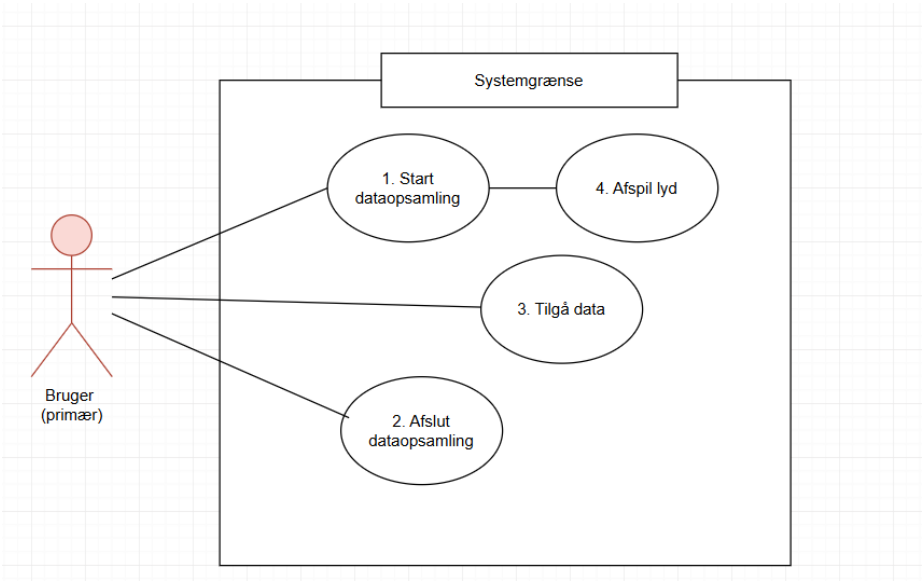
## 2.1 Aktør-kontekst diagram




*Figur 3: Aktør-kontekst diagram*

## 2.2 Use Cases

Interaktionen mellem systemets aktør og Use Cases er afspejlet i Use-Case diagrammet nedenfor. Herefter gives en kort forklaring af hver Use Case. Fully dressed Use Cases kan findes i bilag A afsnit 1.2.5.




*Figur 4 Use-Case diagram*

**Use Case 1: Start dataopsamling:** Denne Use Case beskriver hvordan brugeren starter en badesession og igangsætter registrering af vandforbrug, temperatur og energiforbrug i realtid.

**Use Case 2: Afslut dataopsamling:** Denne Use Case beskriver, hvordan brugeren afslutter en aktiv badesession, hvorefter systemet gemmer data og opdaterer ranglisten.

**Use Case 3: Tilgå data:** Denne Use Case beskriver, hvordan brugeren kan se sine forbrugsdata, estimerede omkostninger og placering på husstandens rangliste.

**Use Case 4: Afspil lyd:** Denne Use Case beskriver, hvordan systemet afspiller et lydsignal, når en forudbestemt mængde vand er blevet forbrugt under en aktiv badesession.

## 2.3 Funktionelle krav

### 2.3.1 Aktører

- **Beboer (Primær aktør):** Husstandsmedlemmet, der tager bad, lader sig nudge af lyde og ønsker at følge med i sin forbrug/rangliste.

### 2.3.2 Tekniske krav og vilkår (Contraints)

**CON-01:** Systemet skal baseres på *embedded* hardware med minimum én Arduino Mega 2560 som den lav-niveau dataopsamlende enhed.

**CON-02:** Systemet skal anvende minimum to sensorer (f.eks. flow og temperatur).

**CON-03:** Løsningen skal indeholde en separat brugergrænseflade, der tillader overvågning.

### 2.3.3 Funktionelle krav

Der defineres en intern prioritering af kravene ved at benytte MoSCoW. MoSCoW-analyse er en metode der danner et overblik over hvilke funktionaliteter der er vigtigst for systemet. Kravene bliver opdelt i følgende kategorier: Must have, Should have, Could have og Won't have (this time). [1]

Den samlede MoSCoW-analyse fremgår af bilag A, afsnit 1.4, herunder de krav der er kategoriseret som *Won't have*. Det fremgår i parentes efter hvert krav nedenfor, hvilken prioritering det enkelte funktionelle krav har.

**FR-01:** Systemet skal måle den præcise mængde af vand, der løber gennem systemet under en badesession, maksimal usikkerhed på < 5 % (MUST).

**FR-02:** Systemet skal løbende måle vandets temperatur (MUST).

**FR-03:** Systemet bør beregne det aktuelle energiforbrug og den tilhørende pris baseret på de opsamlede vand- og temperaturdata (SHOULD).

**FR-04:** Systemet bør afgive et auditivt feedback-signal, hver gang en forudbestemt mængde vand er blevet forbrugt i løbet af en session (SHOULD).

**FR-05:** Systemet bør kunne **præsentere** en rangliste over husstandens badesessioner for at motivere til vandbesparelse (SHOULD).

**FR-06:** Systemet skal kunne identificere den enkelte bruger, så opsamlet data kan knyttes til en personlig profil (MUST).

**FR-07:** Systemet skal i realtid kunne vise det igangværende vandforbrug, energiforbrug og den beregnede pris (MUST).

**FR-08:** Brugeren skal manuelt kunne initiere og afslutte en badesession (MUST).

**FR-09:** Når en session er initieret, skal registrering af forbrug først påbegyndes, når der rent faktisk registreres et vandflow (MUST).

**FR-10:** En badesession kunne kun afsluttes og gemmes korrekt, når vandtilførslen er stoppet (COULD).

## 2.4 Ikke-funktionelle krav

Der er ligeledes foretaget MoSCoW-analyse på de ikke-funktionelle krav. Prioriteringen ses igen i parentes efter kravet.

### 3.4.1 Lyd

**NFR-1.1:** Det auditive signal bør have et lydniveau, der er tydeligt og kan overdøve almindelig baggrundsstøj fra en badedyse (SHOULD).

**NFR-1.2:** Det auditive feedback-signal bør afspilles i realtid (uden mærkbar forsinkelse), præcis når den definerede forbrugsgrænse overskrides (SHOULD).

### 2.4.2 Temperaturmåling og forbrug

**NFR-2.1:** Ved ændring af vandtemperaturen bør systemet registrere og vise den nye, stabile temperatur inden for 5 sekunder (SHOULD).

**NFR-2.2:** Systemet skal måle vandforbruget med en maksimal usikkerhed på < 5 % (MUST).

**NFR-2.3:** Systemets vandførende sensor-komponenter bør være fuldt vandtætte og må ikke lække under normalt, maksimalt vandtryk (SHOULD).

### 2.4.3 Brugergrænseflade

**NFR-3.1:** Brugergrænsefladen skal tillade hurtig og intuitiv navigation. Det må maksimalt kræve ét interaktionstrin at skifte fra en hovedmenu til en direkte undermenu (MUST).

**NFR-3.2:** Systemet bør visuelt og tydeligt indikere over for brugeren, hvor de befinder sig i menu strukturen (SHOULD).

**NFR-3.3:** Præsentationen af data (pris, temperatur, liter) skal være letlæselig på afstand og forståelig for brugeren, mens badet er i gang (MUST).

### 2.4.4 Support og konfiguration

**NFR-4.1:** Systemets softwarearkitektur kunne være struktureret således, at grænseværdien for den auditive feedback (f.eks. antal liter) let kan ændres som en parameter (COULD).

## 2.5 Afgrænsning

Projektet er allerede delvist afgrænset gennem de ovenstående funktionelle og ikke-funktionelle krav ved hjælp af MoSCoW-analysen, hvor kravene er prioriteret efter betydning for systemet.

Projektet afgrænses desuden til udviklingen af et prototypebaseret system til overvågning af vand- og energiforbrug i forbindelse med brusebade i en husstand. Fokus er rettet mod måling af vandforbrug og temperatur, energiberegning samt brugerfeedback gennem visuel præsentation og lydsignaler.

Systemet er derfor ikke designet til permanent installation i et reelt badeværelsesmiljø. Der er f.eks. ikke taget højde for elektrisk sikkerhed, vandtæt indkapsling eller integration med eksisterende VVS-installationer. Skulle systemet integreres i et bademiljø ville sikkerhed mellem elektricitet og vand være et væsentligt fokuspunkt.

Projektet fokuserer på at demonstrere de centrale funktioner og principper bag et overvågningssystem til reduktion af vand- og energiforbrug.

# 3. Metode og proces

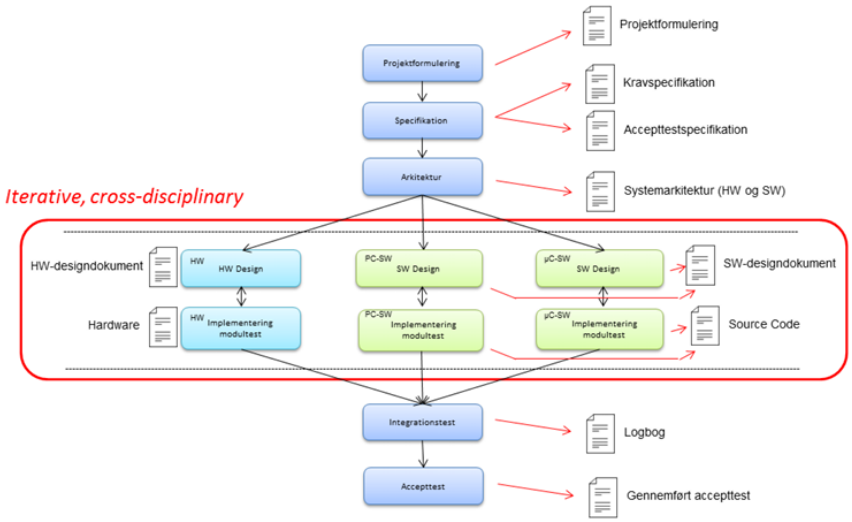
Dette afsnit beskriver gruppens metodebrug og udviklingsproces. Der fokuseres her på ASE-modellen samt anvendelsen af SysML og UML. For et nærmere indblik i gruppedannelse og samarbejde henvises til procesrapporten, bilag H.

## 3.1 ASE-modellen

ASE-modellen er en struktureret udviklingsproces, der benyttes på 2. semester på Aarhus Universitet. Det er en særligt udviklet model, der tager udgangspunkt i vandfaldsmodellen og V-modellen.

V-modellen bygger på sideløbende udvikling af projektets udviklings- og testfaser. Accepttestspecifikationen udarbejdes for eksempel sideløbende med kravsspecifikationen, så man ved, hvordan man skal teste de specificerede krav. Vandfaldsmodellen er derimod baseret på en faseopdelt udvikling, hvor man ikke ændrer på krav, arkitektur og design efter de er blevet specificeret [2]. Den egner sig godt til vores 2. semester projekt, da vi kan specificere krav fra starten og let overskue projektet som helhed.

ASE-modellen kombinerer V-modellens sideløbende udvikling med vandfaldsmodellens ikke-iterative specifikationsproces. ASE-modellen består af en række udviklingsfaser, hvor hver fase har et mål og afsluttes med leveringen af et dokument eller færdigudviklede komponenter. De første 3 faser (se Figur 5) udarbejdes i fællesskab i gruppen og følger en ikke-iterativ proces. Som markeret med rød er det dog tilladt at arbejde iterativt i de næste 3 faser. Disse udarbejdes parallelt og den iterative arbejdsmetode formodes nødvendig da 2.-semesterstuderende ikke er så erfarne inden for systemdesign. Herudover kræver systems engineering ofte løbende erfaring gennem projektforløbet.




*Figur 5: ASE-modellen [2]*

## 3.2 SysML og UML

Et af gruppens vigtigste værktøjer har været SysML og UML. De beskrives i dette afsnit. Siden SysML er en direkte videreudvikling af UML beskrives UML først.

### 3.2.1 UML

UML (Unified Modeling Language) er et modelleringssprog, der anvendes til at beskrive og designe softwaresystemer, særligt systemer med objektorienteret stil. UML-diagrammer bruges til at visualisere hvordan et system er struktureret og opfører sig, så det nemt kan kommunikeres mellem dem der er involveret i projektet.

I projektet er UML anvendt som det, Martin Fowler i *UML Distilled* betegner som en *forward engineering sketch*, hvor diagrammerne udarbejdes inden man skriver koden, så gruppen har mulighed for at diskutere forskellige muligheder og danne et fælles overblik [3, pp. 2-3].

Gruppen har anvendt UML-diagrammer såsom:

- Use case diagram
- Klassediagram
- Sekvensdiagram
- State machine diagram

Flere af diagrammerne er dog udarbejdet i en SysML-inspireret stil, da projektet ikke alene omfatter softwaremodellering, men også modellering af det samlede system.

### 3.2.2 SysML

SysML er et grafisk modelleringssprog, der er videreudviklet fra UML specifikt til systems engineering. SysML har været gruppens foretrukne værktøj, da det muliggør en letforståelig grafisk repræsentation af hele systemet, ikke blot softwareaspekterne. Der er 9 slags SysML diagrammer som hovedsageligt opdeles i 2 hovedtyper, *behavior diagrams* og *structure diagrams* [4, pp. 14-16].

Structure-diagrammer anvendes til at beskrive systemets blokke og komponenter samt relationerne mellem systemets forskellige dele. Diagrammerne giver et overblik over, hvordan systemet er opbygget, og hvordan komponenterne forbindes. Gruppen har udviklet structure-diagrammerne:

- Block definition diagram (BDD)
- Internal block diagram (IBD)

Behavior-diagrammer anvendes til at beskrive systemets dynamiske adfærd under udførelse. Diagrammerne viser blandt andet interaktioner mellem systemets komponenter, rækkefølgen af hændelser samt ændringer i systemets tilstande. Gruppen har udviklet behavior-diagrammerne:

- Use case diagram
- Sekvensdiagram
- State machine diagram

Både SysML- og UML-modellerne har fungeret som et fælles grundlag for udviklingen af systemet og bidraget til en mere struktureret designproces.

# 4. Analyse

## 4.1 Sensorer

Vores projektprodukt er et dataopsamlings-, beregnings- og formidlingsmodul, der kobles op på en eksisterende bruser. Dog, som beskrevet i afgrænsningsafsnittet, har vi holdt os til et prototypesystem, der fungerer ved blot at lade vand løbe gennem en slange, i stedet for at bygge en hel bruser udover projektet. Det har givet nogle flere krav til valg af sensorer.

### 4.1.1 Flowsensor

I kraft af at vi har haft fokus på Smart Shower-modulet og ikke en fysisk bruser, ville vi gerne have en flowsensor, der kunne måle relativt små (ift. almindelige brusere) flowrater (volumen af vand pr. tidsenhed). EmbeddedStock skulle også have sensoren på lager og det skulle være nemt at forbinde sensoren til en vandkilde og fortolke sensorens data.

Der var to sensorer, der var gode kandidater, YF-S401 og YF-S201 baseret på at de begge var tilgængelige på EmbeddedStock, nemme at koble til en vandkilde og nemme at tolke data på.

YF-S401 garanterede præcision på $\pm$5%, hvor YF-S201 garanterede $\pm$10%. Den eneste anden store forskel mellem de to sensorer var, at YF-S401 kan måle flow mellem 0,3-6 L/min med en åbningsdiameter på ¼ inch, hvor YF-S201 måler mellem 1-30 L/min med en åbningsdiameter på ½ inch. For at finde ud af, hvilket "flowrate-lejde", vi burde lede efter, lavede vi en hurtig beregning:

Vores opstilling er en vanddunk, der står på et bord, med en slange, der løber ned i en opsamlingsspand.

Vha. torricellis lov, regnede vi ud, hvad den teoretiske maksimale flowrate, vi kunne regne med at måle, var:

$$v = \sqrt{2gh}$$

Hvor $v$ er vandhastighed, $g$ er tyngdeaccelerationen og $h$ er højden, vandet falder.

$$v = \sqrt{2 \cdot 9{,}81 \frac{m}{s^2} \cdot 0{,}3m} \approx 2{,}43 \frac{m}{s}$$

Det giver en maksimal teoretisk flowrate på:

$$flow = A \cdot v$$

Hvor A er tværsnitsarealet af slangen, vandet løber igennem.

$$A_{S401} = \pi \cdot \left( \frac{\frac{1}{4}}{2}\ in \right)^2 \approx 0{,}3166922\ cm^2, \qquad A_{S201} = \pi \cdot \left( \frac{\frac{1}{2}}{2}\ in \right)^2 \approx 1{,}266769\ cm^2$$

$$flow_{S401} = 0{,}317\ cm^2 \cdot 2{,}43 \frac{m}{s} \approx 4{,}62 \frac{L}{min}$$

$$flow_{S201} = 1{,}27\ cm^2 \cdot 2{,}43\frac{m}{s} \approx 18{,}47\frac{L}{min}$$

Vi vurderede at fordi, det teoretisk maksimale flow for YF-S201 lå tættere på midten af dens opsamlingsvindue, var den bedre til vores projekt end YF-S401. Den eneste forhindring var, at YF-S401 garanterede en bedre målenøjagtighed, men ifølge producenten, kan man opnå lignende eller bedre nøjagtighed med YF-S201 ved korrekt kalibrering. Derfor valgte vi YF-S201.

### 4.1.2 Temperatursensor

Vores krav til temperatursensor var at den skulle være vandtæt, sende måling med digitalt signal, være tilgængelig på EmbeddedStock og dens data skulle være nem at fortolke.

Den eneste temperatursensor på EmbeddedStock var DS18B20. Den sender dog heldigvis data med digitalt, 9-12 bit signal, som er nemme at fortolke.

## 4.2 GUI

Et af projektets krav var, at produktet skulle gøre brug af en GUI. Vi har derfor overvejet hvilken type skærm, vi ville bruge, og hvilket framework og kodesprog, det skulle skrives i.

### 4.2.1 Skærm

Der var to oplagte skærmvalg: 10.1″ HDMI Capacitive Touch Display med Optical Bonding Toughened Glass Panel og den officielle Raspberry Pi 5″ Touch Display 2. Begge understøtter touch og fungerer med Raspberry Pi.

Valget faldt på 10.1″ HDMI-displayet, primært på grund af synlighed og brugervenlighed. Den større skærmstørrelse giver bedre mulighed for at præsentere information tydeligt og gør det lettere for brugeren at aflæse data under drift. Dette vurderedes særligt vigtigt i projektet, hvor flere informationer skal kunne vises samtidig uden at skabe et uoverskueligt brugerinterface.

### 4.2.2 Software framework

Til GUI valgte vi Flask, et letvægtswebframework i Python, som kører på Raspberry Pi og vises i en browser på det tilkoblede display. Da projektet allerede havde en C++ kerne, var et centralt spørgsmål, hvordan GUI og logik skulle kommunikere. Flask var ikke et valg baseret på forhåndskendskab, vi vurderede dog det vil være en simplere indgang til GUI-udvikling end fx Qt, da HTML og CSS gør det forholdsvist hurtigt at opbygge og style sider, og Python kræver mindre opsætning end et C++ framework. Alternativet med Qt ville i teorien have forenklet kommunikationen, da GUI og logik kunne dele objekter direkte frem for at kommunikere via tekstfiler, men Qt er et tungt framework med en stejl indlæringskurve.

| Løsning | Fordele | Ulemper |
|---|---|---|
| Flask | - Begyndervenlig, godt nu hvor det første gang gruppen arbejder med GUI<br>- HTML/CSS gør styling forholdsvist simpelt | - Kræver tekstfiler til kommunikation med C++<br>- Ekstra kommunikationslag i arkitekturen |
| Qt | - Direkte adgang til C++ objekter.<br>- Alt kører i samme sprog og miljø | - Tungt framework med stor opsætning (bruger UI frem for HTML)<br>- Ikke begynder venligt |

*Tabel 1 - Fordele og ulemper med GUI*

Som det ses på tabellen, er der fordele og ulemper ved begge løsninger. Flask blev valgt da det vurderedes til en mere begyndervenlig løsning, og det var første gang nogen af os havde arbejdet med et graphic user interface. Så på trods af at Qt ville have forenklet kommunikationen mellem GUI og C++ valgte vi Flask. Valget betød at vi måtte acceptere et ekstra kommunikationslag via tekstfiler i arkitekturen.

# 5. Arkitektur

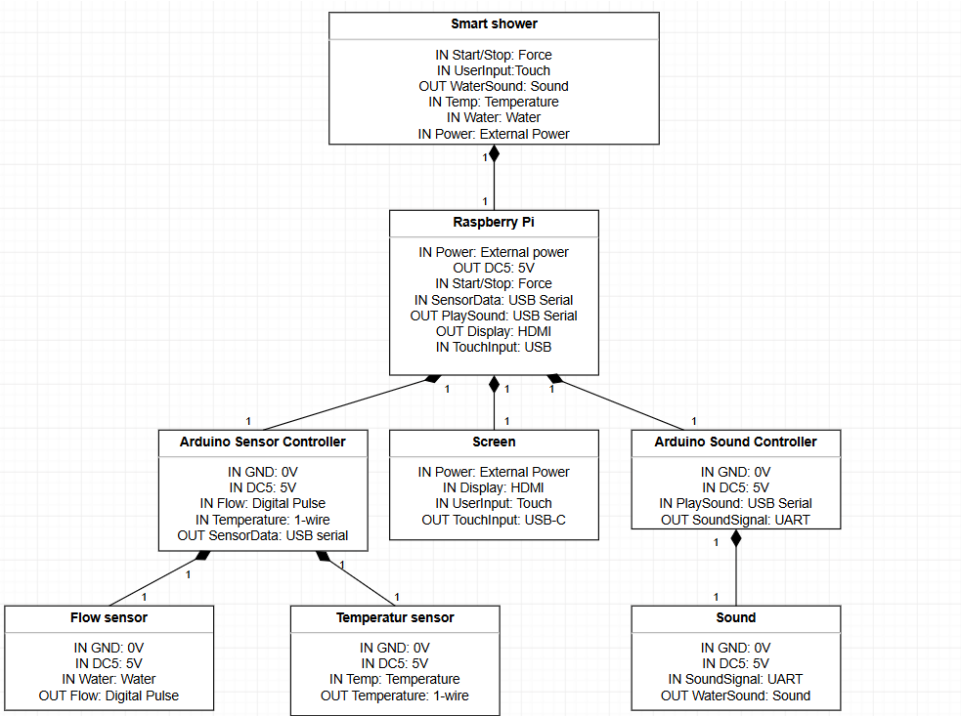
I dette afsnit beskrives systemarkitekturen for vores *Smart Shower*. Arkitekturen er opdelt i et hardware- og softwareafsnit. Systemarkitekturen beskriver systemets grundlæggende opbygning og funktioner, og danner rammen for udviklingen af systemet.

## 5.1 Hardwarearkitektur

I dette afsnit beskrives hardwarearkitekturen for systemet. Et Block Definition Diagram (BDD) illustrerer systemets hardwareblokke og deres sammenhæng. Derudover anvendes et Internal Block Diagram (IBD) til at vise forbindelserne og kommunikationen mellem blokkene.

### 5.1.1 BDD

Figur 6 viser systemets overordnede hardwarearkitektur gennem et Block Definition Diagram (BDD). Diagrammet illustrerer systemets centrale hardwareblokke samt deres forbindelser og signaludveksling.




*Figur 6: BDD*

Der benyttes to Arduino-enheder i systemet: én til sensordata og én til lydstyring. Opdelingen er valgt for at undgå konflikter mellem de to funktioner. Begge Arduinoer forbindes til Raspberry Pi'en, som leverer strøm, mens Arduinoerne selv forsyner deres tilkoblede komponenter. Vi havde først planlagt blot at bruge én Arduino, men indså at dette kunne påvirke vores kommunikation mellem Raspberry Pi og Arduinoen.

Der er udarbejdet en blokbeskrivelse, der giver en kort forklaring af de enkelte komponenters ansvar og samspil i den samlede hardwarearkitektur (se Tabel 2 nedenfor).

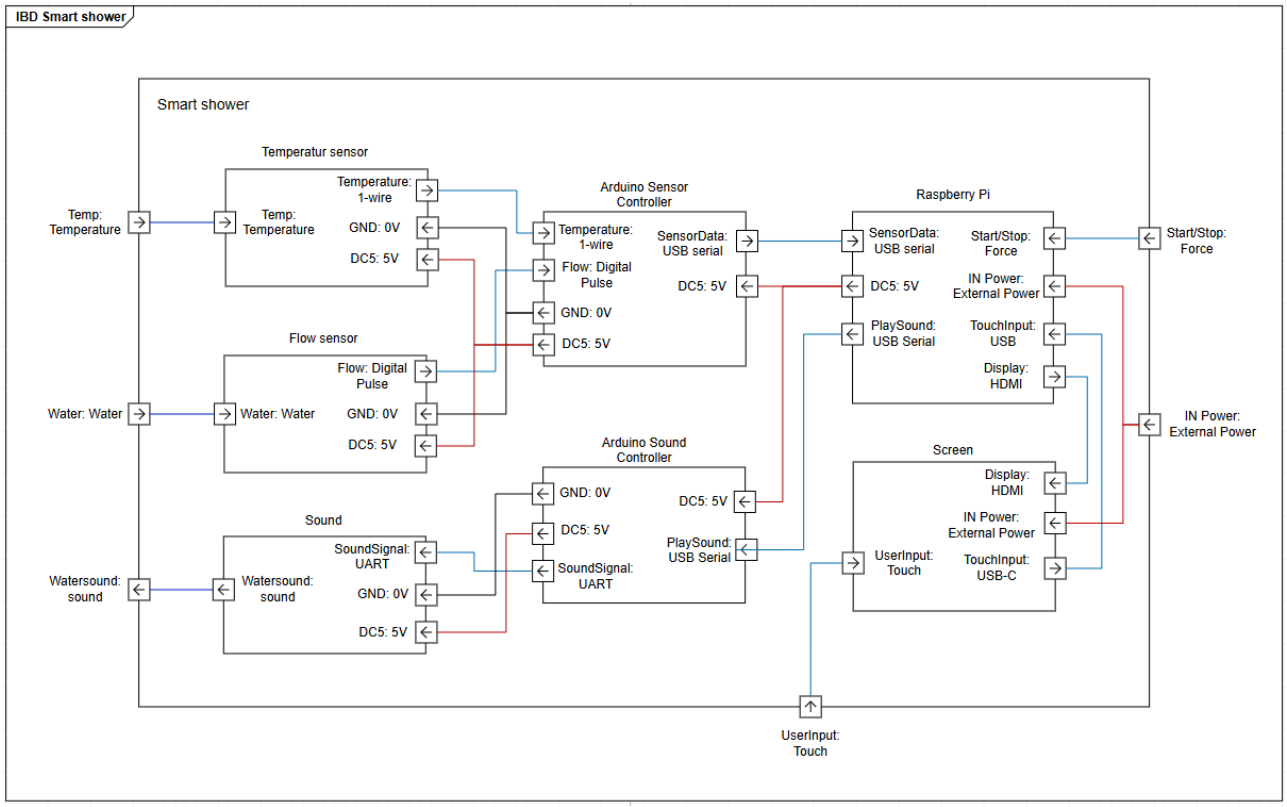
| Blok navn | Funktionsbeskrivelse | Signaler | Kommentar |
|---|---|---|---|
| Flow sensor | Måler vandmængden i systemet og sender måledata til Arduino | IN GND: 0V | Fælles stel |
| | | IN DC5: 5V | Spændingsforsyning |
| | | IN Water: Water | Vand ind i sensoren |
| | | OUT Flow: Digital Pulse | Sender flowdata til Arduino |
| Temperatur sensor | Måler vandets temperatur og sender temperaturdata til Arduino | IN GND: 0V | Fælles stel |
| | | IN DC5: 5V | Spændingsforsyning |
| | | IN Temp: Temperature | Temperatur ind i sensoren |
| | | OUT Temperature: 1-wire | Sender temperaturdata til Arduino |
| Screen | Brugergrænsefladen der viser menuer og statistik til brugeren samt registrerer brugerens input | IN GND: 0V | Fælles stel |
| | | IN DC5: 5V | Spændingsforsyning |
| | | IN Display: HDMI | GUI fra Raspberry Pi |
| | | IN UserInput: Touch | Touch-input fra bruger |
| | | OUT TouchInput: USB-C | Sender brugerinput til Raspberry Pi |
| Raspberry Pi | Systemets hovedcontroller. Modtager sensordata fra Arduino, beregner vand- og energiforbrug samt pris, håndterer brugere og ranglister, opdaterer skærmen og aktiverer sound. Forsynes af en ekstern strømforsyning. Forsyner begge Arduinoer med 5V. | IN External Power | Ekstern strømforsyning |
| | | OUT DC5: 5V | Forsyner tilsluttede Arduino-enheder med 5 V DC |
| | | IN Start/Stop: Force | Fysisk knap der trykkes på |
| | | IN SensorData: USB Serial | Sensor data fra Arduino |
| | | OUT PlaySound: USB Serial | Sender signal om at lyd skal afspilles til Arduino Sound Controller |
| | | OUT Display: HDMI | Sender GUI til Screen |
| | | IN TouchInput: USB | Brugerinput fra Screen |
| Arduino Sensor Controller | Processerer input fra Flow- og Temperatur sensor og leverer dem som output til Raspberry Pi. Forsynes med 5V. | IN GND: 0V | Fælles stel |
| | | IN DC5: 5V | Spændingsforsyning |
| | | IN Flow: Digital Pulse | Flowdata fra Flow sensor |
| | | IN Temperature: 1-wire | Temperaturdata fra Temperatur sensor |
| | | OUT SensorData: USB Serial | Sensor data til Raspberry Pi |
| Arduino Sound Controller | Kommunikerer mellem Raspberry Pi og Sound blokken. Når en lyd skal afspilles, sender Raspberry Pi et signal til Sound gennem Arduino Sound Controller. | IN GND: 0V | Fælles stel |
| | | IN DC5: 5V | Spændingsforsyning |
| | | IN PlaySound: USB Serial | Signal fra Raspberry Pi, sendes videre til Sound |
| | | OUT SoundSignal: UART | Signal til Sound, sendes når der skal afspilles en lyd. |
| Sound | Afspiller lydsignal når en forudbestemt vandmængde er | IN GND: 0V | Fælles stel |
| | | IN DC5: 5V | Spændingsforsyning |

| | | | |
|---|---|---|---|
| | nået, som feedback til brugeren under badet | IN SoundSignal: UART | Signal fra Arduino Sound Controller |
| | | OUT WaterSound: Sound | Lyd til brugeren |

*Tabel 2: Blokbeskrivelse*

### 5.1.2 IBD

Figur 7 viser systemets interne forbindelser gennem et Internal Block Diagram (IBD). Diagrammet illustrerer signalerne mellem systemets hardwareblokke samt hvordan de enkelte komponenter er forbundet internt i systemet.




*Figur 7: IBD af Smart Shower*

IBD'et viser at vores flow- og temperatursensor er forbundet til Arduino Sensor Controller, som opsamler sensordata og videresender dem til Raspberry Pi'en via USB-seriel kommunikation. Raspberry Pi'en fungerer som systemets centrale enhed og håndterer både brugerinput fra touchskærmen og visning af data på skærmen via HDMI.

Systemets lydfunktion håndteres af en separat Arduino Sound Controller, som modtager kommandoer fra Raspberry Pi'en og sender lydsignaler videre til lydmodulet via UART-kommunikation. Derudover viser IBD'en også strømforsyningen mellem komponenterne i systemet.

For en mere detaljeret beskrivelse af signalerne henvises til tabel 2 i bilag C

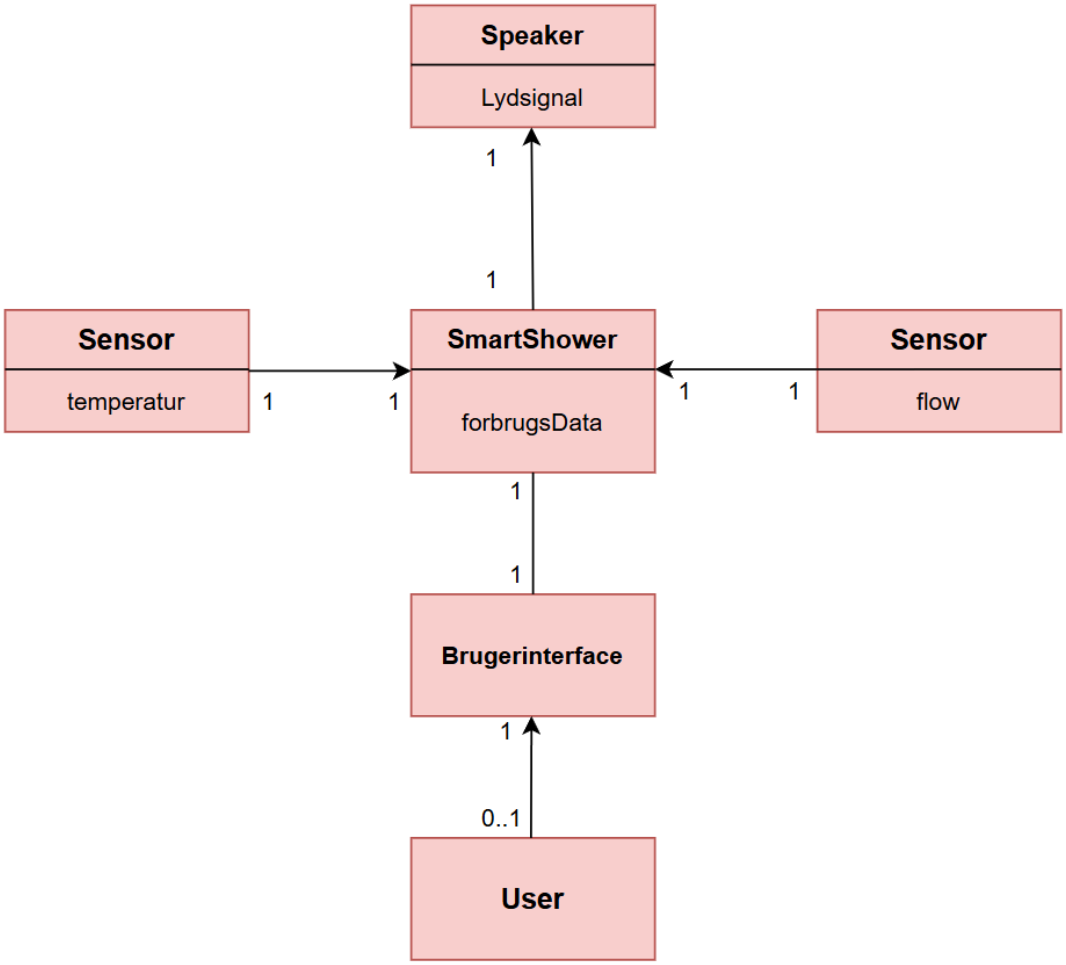
## 5.2 Softwarearkitektur

Softwarearkitekturen beskrives ved brug af en domænemodel samt beskrivelse af udvalgte applikationsmodeller.

### 5.2.1 Domænemodel

Domænemodellen beskriver systemets centrale begreber og relationer og danner grundlag for den efterfølgende softwarearkitektur.

Modellen bygger på en navneordsanalyse af kravspecifikationen og use cases, hvor relevante begreber og relationer er identificeret. Denne kan findes i afsnit 2.1.1 bilag C

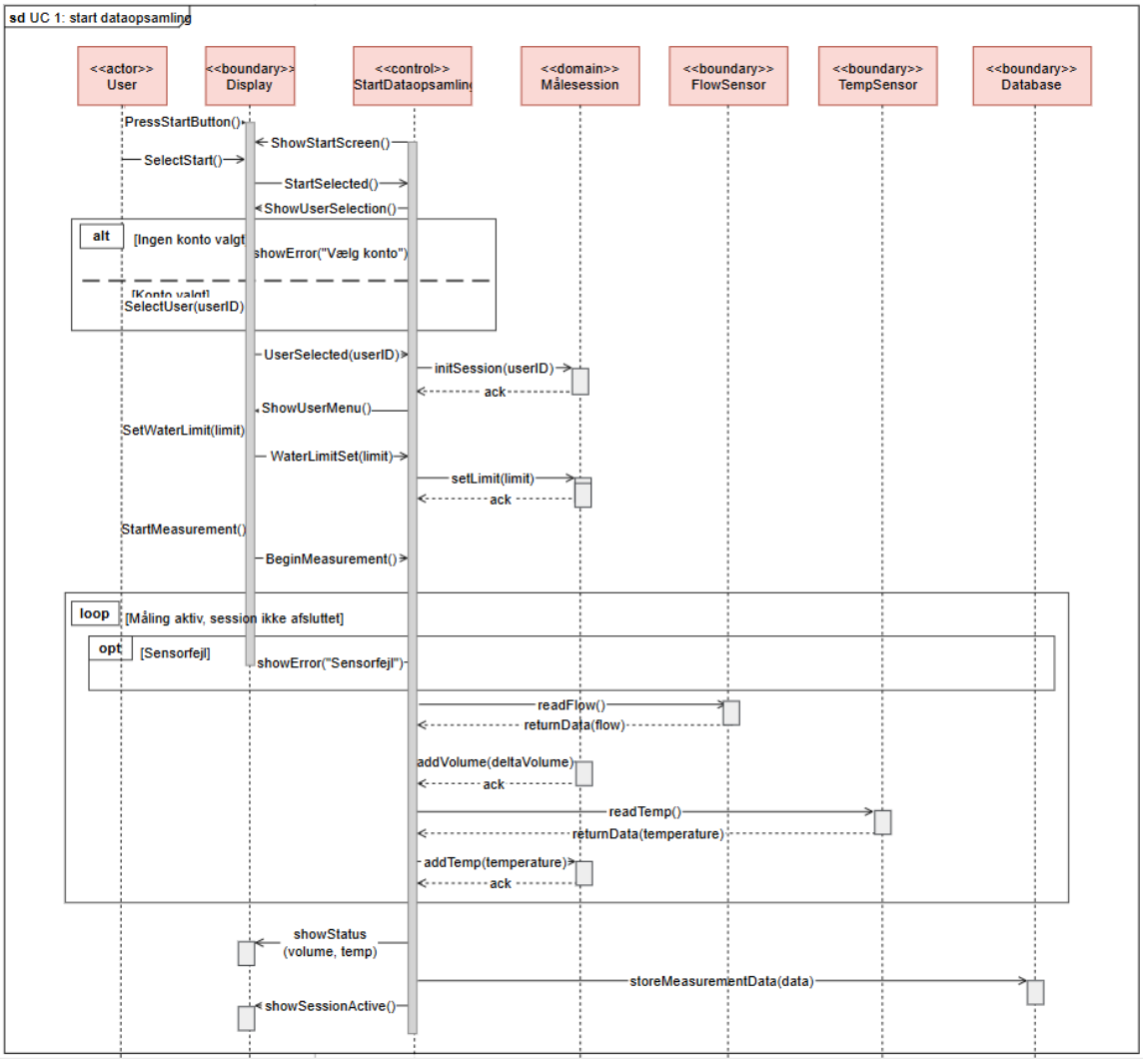



*Figur 8: Domænemodel for Smart Shower*

### 5.2.2 Applikationsmodeller

Med udgangspunkt i domænemodellen og de beskrevne Use Cases i kravsspecifikationen udarbejdes en applikationsmodel for hver Use Case. Modellerne er udarbejdet ud fra et Entity-Control-Boundary pattern, hvor ansvaret opdeles mellem brugergrænseflade, kontrolklasser og domæneklasser [5].

En applikationsmodel består af et *sekvensdiagram* (SD), et *klassediagram* og et *state machine diagram* (STM). Figur 9 nedenfor viser sekvensdiagrammet udarbejdet for UC 1: Start dataopsamling. Netop dette sekvensdiagram er udvalgt, da det beskriver systemets centrale funktionalitet og illustrerer kommunikationen mellem brugergrænseflade, kontrolklasse, sensorer og database under en aktiv målesession.



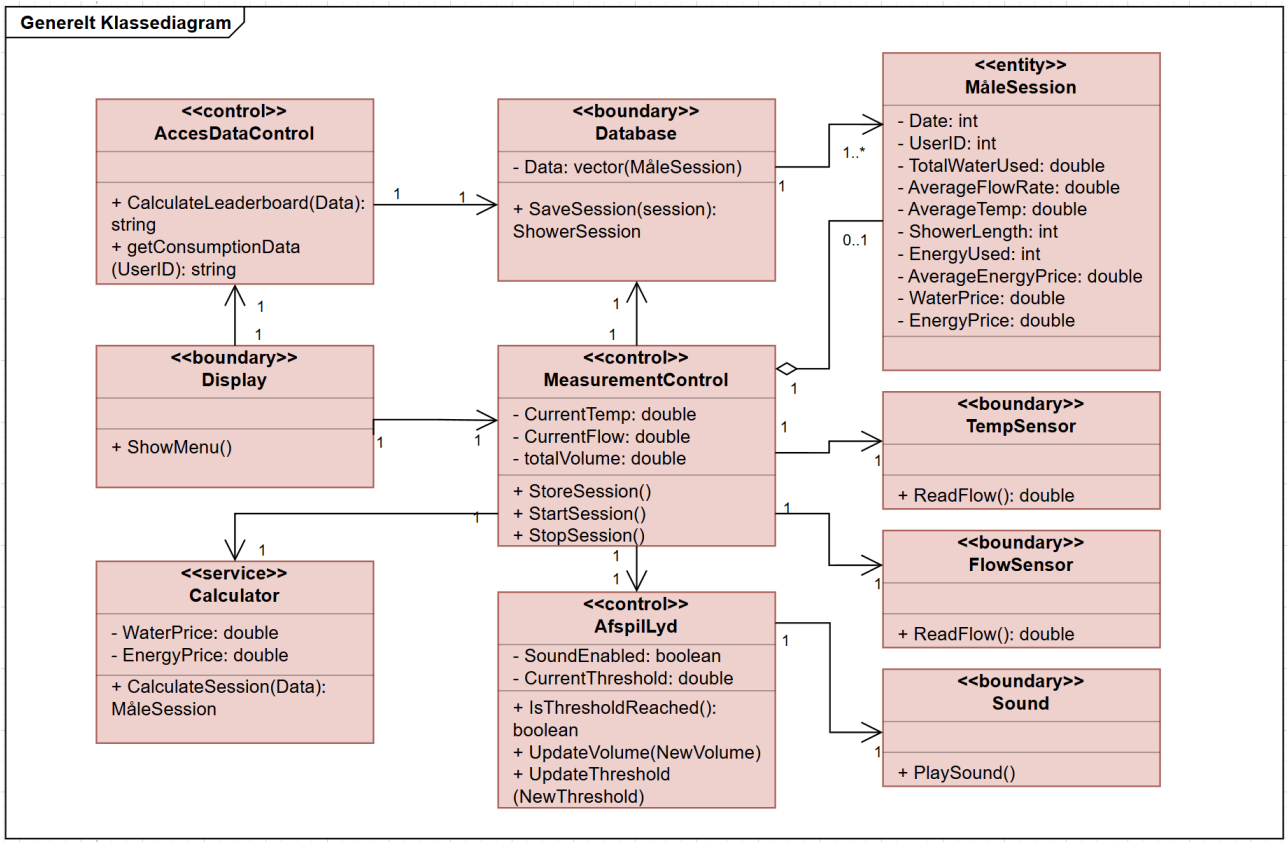

*Figur 9: Sekvensdiagram for UC 1*

Diagrammet viser blandt andet brugerens valg af konto, opsætning af vandgrænse samt systemets løbende registrering og lagring af måledata. Derudover illustrerer diagrammet anvendelsen af loop-, alt- og opt-fragmenter til håndtering af gentagne sensoraflæsninger og fejlscenarier under

dataopsamlingen. For en komplet gennemgang af alle applikationsmodeller henvises der til afsnit 2.2 i Bilag C.

Figur 10 viser det generelle UML-klassediagram udarbejdet for systemet. Klassediagrammet giver et overordnet indblik i systemets softwarestruktur samt relationerne og associationerne mellem klasserne. Her illustreres blandt andet, hvordan *MeasurementControl* kommunikerer med sensor-klasserne *TempSensor* og *FlowSensor* ved indsamling af måledata, samt hvordan data lagres i *Database*.

Det generelle klassediagram fungerer som en samlet oversigt over systemets softwarearkitektur og danner grundlag for den efterfølgende design- og implementeringsfase.




*Figur 10: Generelt klassediagram*

# 6. Design og implementering

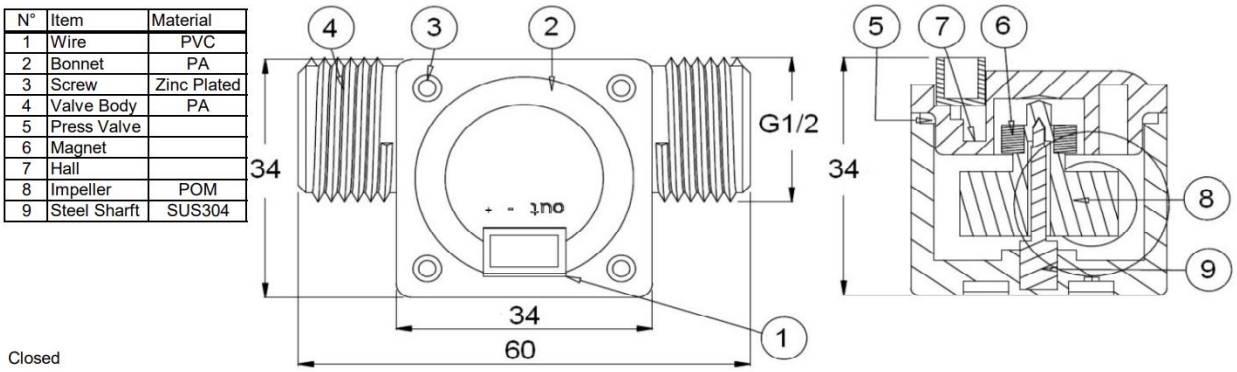
## 6.1 Hardware design

### 6.1.1 Flow sensor

Til måling af vandmængden under en badesession anvendes en YF-S201 Hall-effekt-flowsensor, som er designet til at måle vandgennemstrømningen i en slange. Sensoren er valgt på baggrund af dens lave strømforbrug, egnede måleområde og dens simple tilslutning til mikrocontrolleren (jf. afsnit 4.1.1)

YF-S201 består af et plastikhus, en vandrotor og en Hall-effekt-sensor. Når vandet strømmer igennem, begynder vandrotoren at rotere med en hastighed, der er proportional med vandstrømmen. En magnet monteret på rotoren passerer Hall-effekt-sensoren ved hver omdrejning, hvilket genererer én puls for hver omdrejning. Den genererede pulsfrekvens er derfor direkte proportional med den vandmængde, der strømmer igennem sensoren.

Hardwaremæssigt forbindes flowsensorens datasignal direkte til Arduinoens Pin 2 (Port E, pin 4), da denne understøtter hardware-interrupts (External Interrupt 4). For at sikre et stabilt signal uden ”floating”-tilstande (svævende input) aktiveres Arduinoens indbyggede, interne pull-up-modstand på den valgte pin. Signalet aflæses herefter pålideligt via hardware-interruptet, der reagerer på det stigende signal (*rising edge*), når magneten passerer.



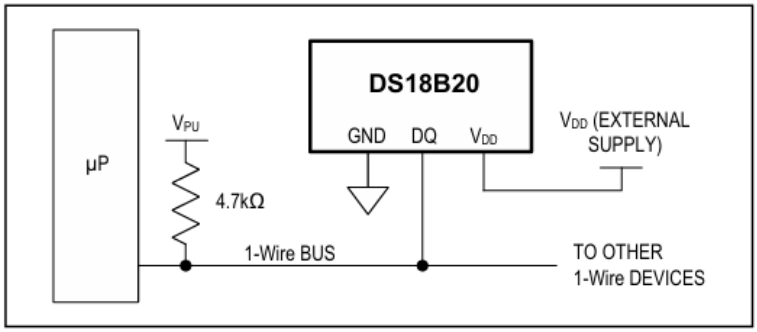

*Figur 11: Arkitektur på YF-S201 flowsensor [6]*

### 6.1.2 Temperatur sensor

Til at måle temperaturen af vandet benytter vi en 1-Wire DS18B20 temperatursensor. Vi har valgt denne sensor, da den er vandtæt og kun kræver en dataforbindelse (jf. afsnit 4.1.2). Det fremgår af databladet, at sensoren kommunikerer med en open-drain udgang, hvilket kræver en pull-up modstand på 4,7 kΩ for at holde databussen i logisk høj hviletilstand. Mikrocontrolleren kan herefter trække linjen lav for at initiere kommunikation og modtage temperaturdata. [7]

Hardwareopkoblingen af en DS18B20 kan ses på figuren nedenfor. Sensoren forsynes med 5V fra Arduinoens eksterne strømforsyning frem for at benytte parasitic power mode. Dette valg giver en mere stabil drift og reducerer risikoen for kommunikationsfejl.

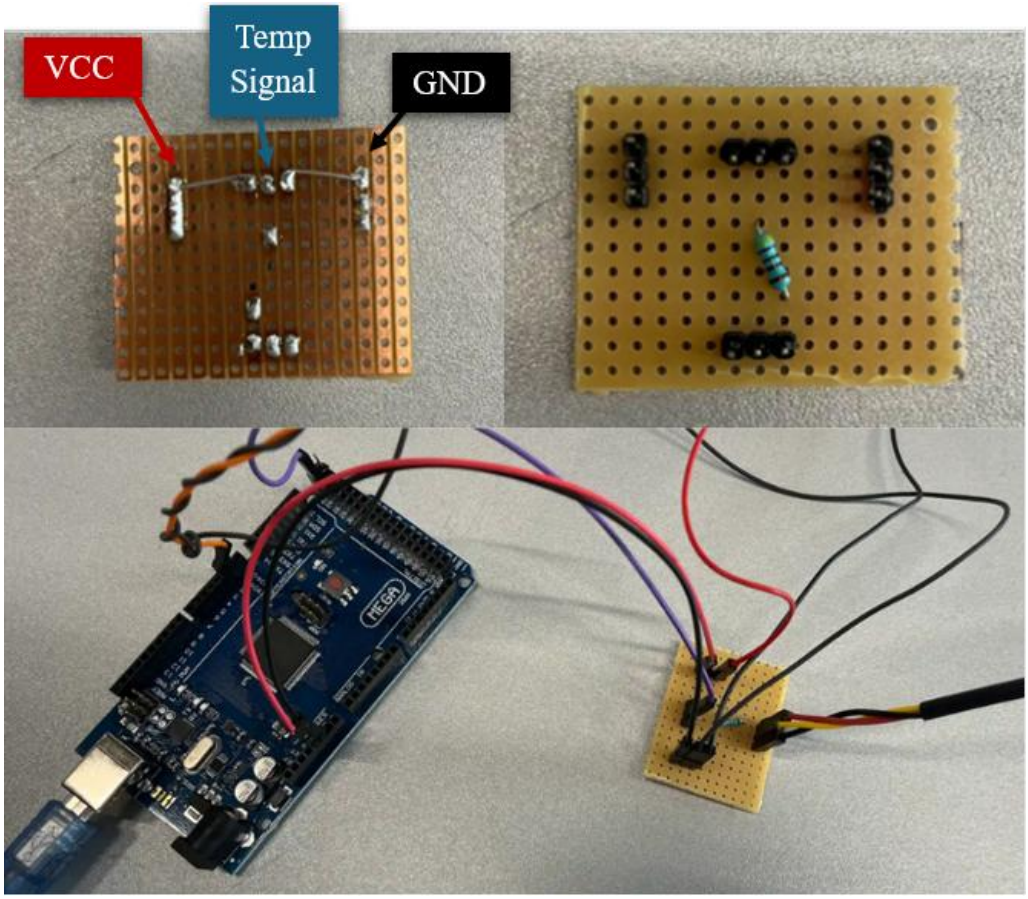
Sensorens datasignal forbindes til Arduinoens pin 22 (Port A, pin 0). Denne pin er valgt, da 1-Wire kræver en digital input/output-pin til kommunikation mellem sensoren og microcontrolleren. Temperaturen kan herefter aflæses ved, at Arduinoen styrer kommunikationen over databussen.




*Figur 12: Hardwareopkobling af DS18B20 med ekstern strømforsyning (VDD), figur 7 i databladet. [7]*

### 6.1.3 Implementering af sensorer

Implementeringen af temperatur- og flowsensoren var forholdsvis simpel. Flowsensoren blev forbundet direkte til Arduinoens VCC, GND og signalpin, mens temperatursensoren krævede en 4,7 kΩ pull-up modstand mellem signal og VCC. For at give begge sensorer fælles VCC og GND blev forbindelserne samlet på et VERO-board med separate baner til 5V og ground, som vist på Figur 13. Pull-up modstanden blev loddet mellem VCC-banen og temperatursensorens signalbane, hvorefter begge sensorer blev tilsluttet Arduinoen til dataopsamling.




*Figur 13: Implementering af temperatur- og flowsensor*

### 6.1.4 Lyd

Lydsystemet består af følgende hovedkomponenter:

- Arduino ATmega2560
- SOMO-II lydmodul
- MicroSD kort
- Højttaler (3 W)

Til lydafspilning anvendes SOMO-II lydmodulet, som kan afspille MP3, filer direkte fra et micro-SD kort formateret i FAT32. Modulet er valgt, fordi det selv håndterer afkodning og afspilning af MP3, hvilket aflaster mikrocontrolleren. Kommunikationen sker via en UART-protokol, hvor ATmega2560 sender 8 byte kommandopakker, som beskrevet i SOMO-II databladet.

En 3 W højttaler er tilsluttet SOMO-II's lydudgang og anvendes til at afspille lydene. Lydstyrken styres softwaremæssigt via kommandoer fra ATmega2560 til SOMO-II. SPK+ forbindes til ben 15 på SOMO-II og SPK- forbindes til ben 16 på SOMO-II.

SOMO-II og ATmega2560 forsynes med en spænding på 5 V strømforsyning. VCC går ind på ben 9 på SOMO-II. Der anvendes fælles stel (GND). GND går ind på ben 10 på SOMO-II.

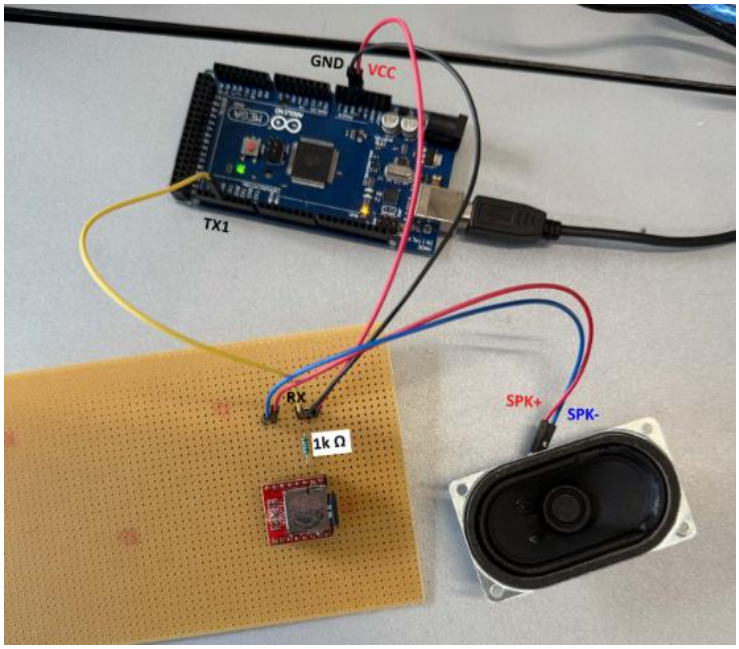
Seriel kommunikation (UART) Arduino TX1 forbindes til SOMO-II RX. Mellem TX1 og RX anvendes en 1 k ohm seriemodstand, som henvist af SOMO-II databladet. Modstanden begrænser strømmen og reducerer belastningen på RX-indgangen, da Arduinoens TX arbejder på 5 V, mens SOMO-II's RX er 3,3 V.

UART-indstillingerne skal matche SOMO-II og er fastlagt ud fra databladet:

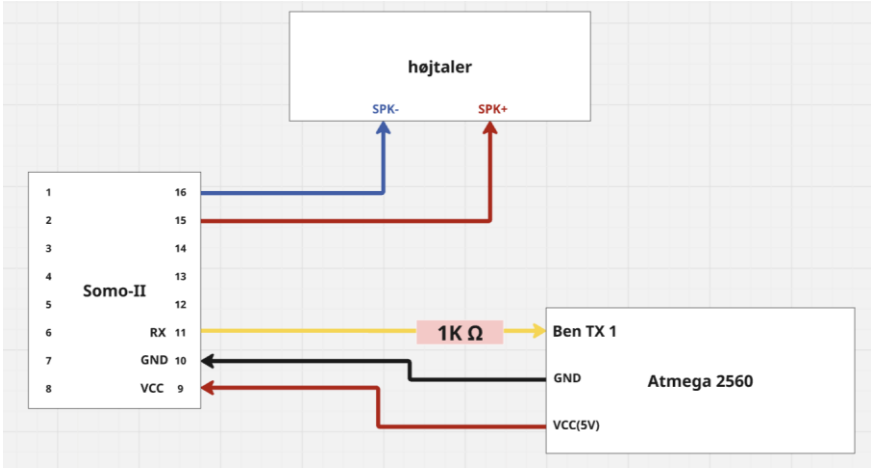
- Baudrate: 9600
- 8 databits
- Ingen paritet
- 1 stopbit

Baudraten beregnes for ATmega2560 ved 16 MHz med formlen: UBRR = (F_CPU / (16 × baud)) − 1. Med F_CPU = 16.000.000 Hz og baud = 9600 fås UBRR = (16.000.000 / (16 × 9600)) − 1 = 103,17, som rundes til 103. Der indlæses derfor UBRR1H = 0x00 og UBRR1L = 0x67.

Nedenfor ses et billede af vores færdige lyd implementering samt en model over hvordan de er forbundet, som giver et overblik over lydens hardwaresammensætning.




*Figur 14: Implementering af lyd*

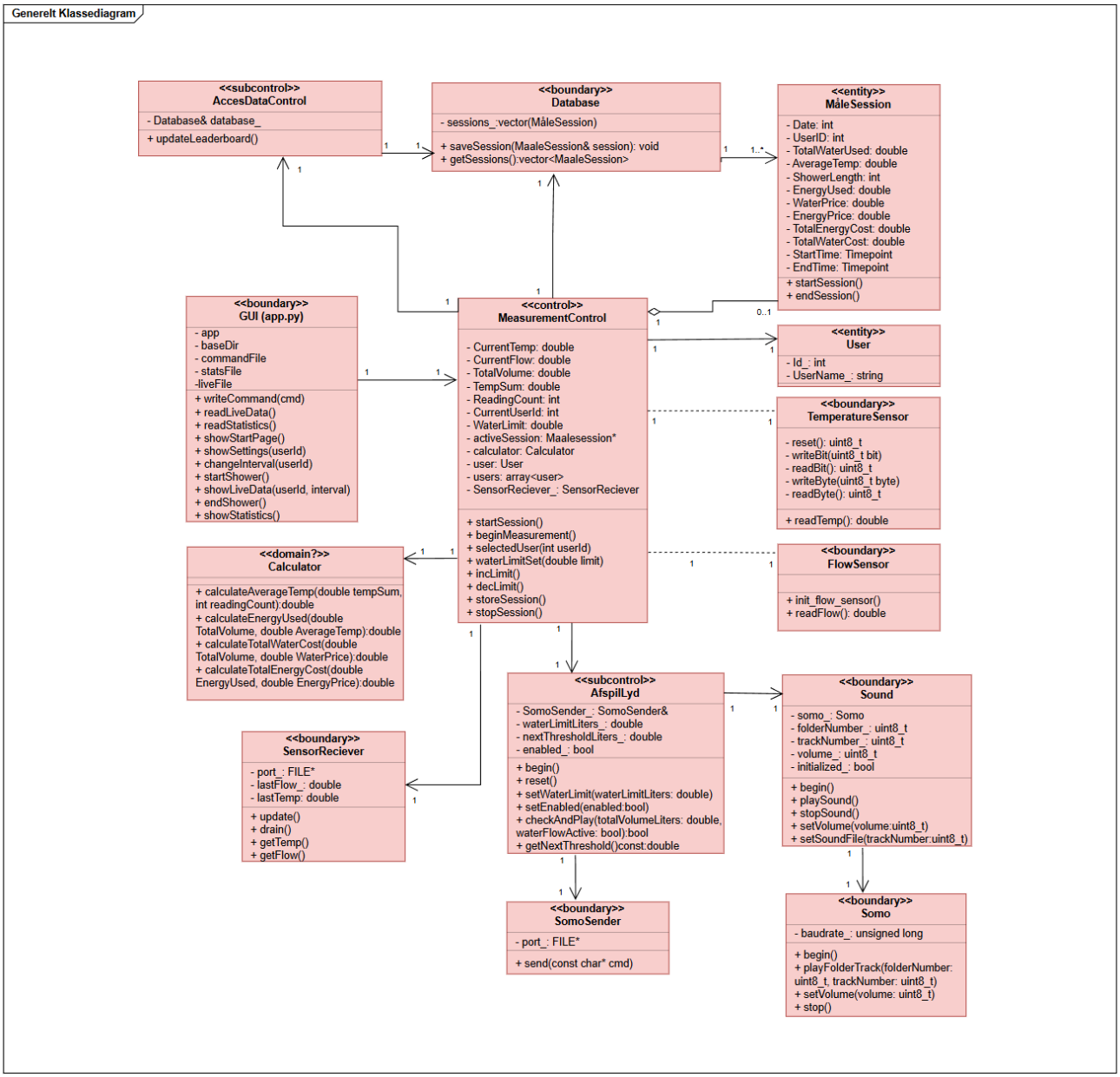



*Figur 15: Overblik over lyd hardware*

## 6.2 Software design

Følgende afsnit beskriver design og implementering af systemets centrale klasser, hvor de vigtigste metoder i hver klasse beskrives. Beskrivelse af underliggende klasser, som SomoSender og SensorReceiver, samt test af de enkelte klasser er udeladt i rapporten, der henvises i stedet til afsnit 1.2 i bilag D.

På baggrund af vores SW-design er der udarbejdet et opdateret klassediagram.




*Figur 16: Opdateret klassediagram*

### 6.2.1 Calculator

#### 6.2.1.1 Beskrivelse af metoder

Når *MeasurementControl* skal gemme et *Maalesession*-objekt, skal der udføres nogle beregninger på de opsamlede rådata for den aktive session, så hvert *Maalesession*-objekt indeholder alle relevante data for slutbrugeren. Disse beregninger udføres af *Calculator*-klassen, der kaldes af *MeasurementControl*.

Først beregnes den aktive sessions gennemsnitlige vandtemperatur. *MeasurementControl* beregner løbende summen af alle temperaturmålingerne, den foretager og tæller antallet af målinger. Det simple gennemsnit for alle målingerne beregnes derved:

```
5   double Calculator::calculateAverageTemp(double tempSum, int readingCount) {
6       if (readingCount == 0) {
7           return 0.0;
8       }
9       return tempSum / readingCount;
10  }
```

*Figur 17: kode udsnit, calculateAverageTemp, Calculator*

Hver *Maalesession* indeholder også det samlede energiforbrug. Denne findes ved udregning af den energi, det tager at opvarme den brugte vandmængde fra 8 °C (grundvandets temperatur) til badets gennemsnitstemperatur. Vi bruger en værdi på vandets specifikke varmekapacitet på 4186 J/(kg×°C). Vi antager desuden at en liter vand vejer 1 kg. Slutteligt divideres udregningen med 3600000 J/kWh for at konvertere tallet fra Joule til kWh.

```
12  double Calculator::calculateEnergyUsed(double TotalVolume, double AverageTemp) {
13      int SpecifikVarmeKapacitet = 4186;
14      int JouleToKWh = 3600000;
15      int DeltaTemp = AverageTemp - 8;
16      return (TotalVolume * SpecifikVarmeKapacitet * DeltaTemp) / JouleToKWh;
17  }
```

*Figur 18: kode udsnit, calculateEnergyUsed, Calculator*

Den samlede omkostning for vandforbruget er en simpel beregning, hvor mængden af vandet ganges med prisen for en liter vand. *Maalesession*-objektet er født med en vandpris-sats, vi har bestemt til 0,065 kr. pr. Liter. [8]

```
19  double Calculator::calculateTotalWaterCost(double TotalVolume, double WaterPrice) {
20      return TotalVolume * WaterPrice;
21  }
```

*Figur 19: kode udsnit, calculateTotalWaterCost, Calculator*

Ligeledes er den samlede energiomkostning en simpel beregning, hvor vi ganger den brugte energimængde med prisen for en kWh energi. *Maalesession*-objektet er født med en energipris-sats,

vi har bestemt til 2,5 kr. pr. kWh. [9]

```
double Calculator::calculateTotalEnergyCost(double EnergyUsed, double EnergyPrice) {
    return EnergyUsed * EnergyPrice;
}
```

*Figur 20: kode udsnit, calculateTotalEnergyCost, Calculator*

### 6.2.2 FlowSensor

#### 6.2.2.1 Beskrivelse af klassen:

FlowSensor er en boundary-klasse, der håndterer den direkte kommunikation med den fysiske YF-S201 flowsensor. Klassens primære ansvarsområde er at opsamle digitale pulser fra sensoren ved hjælp af hardware-interrupts (ISR) og omsætte disse pulser til en målbar vandvolumen. For at sikre, at der ikke mistes data under hovedprogrammets afvikling, benyttes en global volatile variabel, som tæller op hver gang sensorens Hall-effekt modul registrerer en omdrejning på vandrotoren. [6]

```
#pragma once

class FlowSensor {
public:
    void init_flow_sensor();
    double readFlow();
    void volumeUpdated(double deltaVolume);

};
```

*Figur 21: kodeudsnit af FlowSensor klassen*

#### 6.2.2.2 Beskrivelse af metoder:

**init_flow_sensor()**

**Returtype:** void

**Parametre:** Ingen

**Beskrivelse af funktionalitet:** Denne metode står for opsætningen af mikrocontrollerens hardware-registre til at modtage data fra sensoren. Pin E4 (Pin 2) konfigureres som et input, og den interne pull-up modstand aktiveres for at sikre et stabilt signal. Derefter konfigureres External Interrupt 4 (INT4) til at trigge på en "rising edge" (når signalet går fra 0V til 5V). Slutteligt aktiveres interruptet, og den globale pulstæller (flowPulseCount) nulstilles for at sikre et rent udgangspunkt inden en ny måling.

```
void FlowSensor::init_flow_sensor() {
    // pin 2 som input
    DDRE &= ~(1 << DDE4);

    // intern pull-up modstand på Pin 2
    PORTE |= (1 << PORTE4);

    // Konfigurer External Interrupt 4 (Pin 2) til at trigge på "rising edge" (når signalet går f
    EICRB |= (1 << ISC41) | (1 << ISC40);

    // Aktiver External Interrupt 4, så Arduinoen lytter på Pin 2
    EIMSK |= (1 << INT4);

    // Nulstil tælleren for at være sikker på en ren start
    flowPulseCount = 0;
}
```

*Figur 22: kodeudsnit af init_flow_sensor() metoden*

**readFlow()**

**Returtype:** double

**Parametre:** Ingen

**Beskrivelse af funktionalitet:** Metoden udregner og returnerer den mængde vand, der er strømmet gennem sensoren siden sidste kald. For at undgå *race conditions*, hvor et interrupt ændrer variablen mens den læses, deaktiveres interrupts kortvarigt med cli(), hvorefter den aktuelle værdi af flowPulseCount kopieres og nulstilles, inden interrupts genaktiveres med sei(). Herefter omregnes pulserne til et flow i liter per minut ved hjælp af sensorens kalibreringsfaktor (division med 9.9). Dette flow omregnes derefter til en specifik volumen (deltaVolume) ved at dividere med 60.0. Den beregnede volumen returneres til det overordnede kontrol-lag.

```
double FlowSensor::readFlow() {
    cli();
    uint16_t pulses = flowPulseCount;
    flowPulseCount = 0;
    sei();

    // Beregner flow i liter per minut
    double currentFlowRate = pulses / 9.9;

    double deltaVolume = currentFlowRate / 60.0;

    return deltaVolume;
}
```

*Figur 23: kodeudsnit af readFlow() metoden*

### 6.2.3 TemperatureSensor

#### 6.2.3.1 Beskrivelse af klassen:

TemperatureSensor er en boundary-klasse som håndterer kommunikationen med temperatursensoren DS18B20. Klassen har en enkelt public funktion *readTemp()*, som returnerer en double med temperaturen i celsius. For at implementere *readTemp()* benyttes fem private hjælpefunktioner: *reset()*, *writeBit(uint8_t bit)*, *readBit()*, *writeByte(uint8_t)*, *readByte()*.

```
class TemperatureSensor {
public:
    double readTemp();

private:
    uint8_t reset();
    void writeBit(uint8_t bit);
    uint8_t readBit();
    void writeByte(uint8_t byte);
    uint8_t readByte();
};
```

*Figur 24: Kodeudsnit af TemperatureSensor.h*

#### 6.2.3.2 Beskrivelse af metoder:

**reset():**

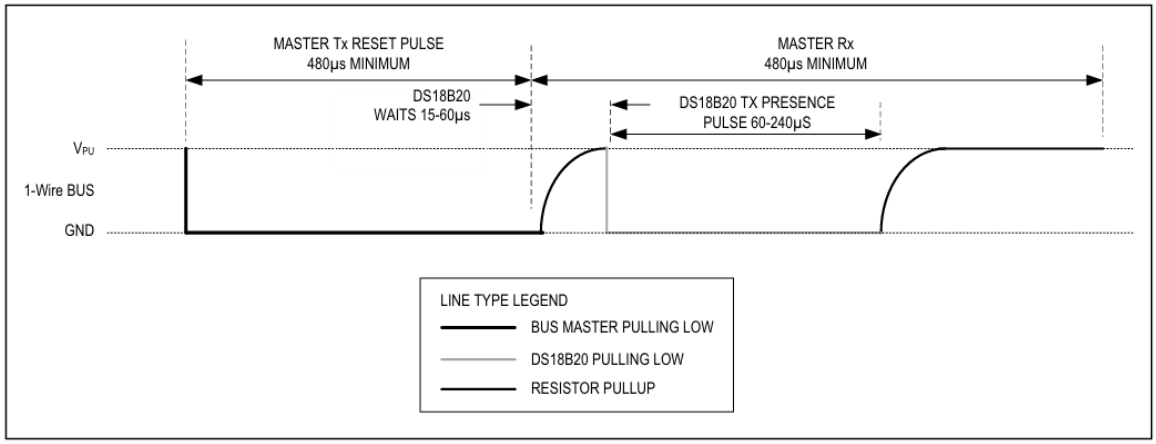
**Returtype:** uint8_t

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Ifølge databladet skal al kommunikation med sensoren starte med et reset-pulse fra masteren efterfulgt af et presence-pulse fra sensoren. Timingen af dette kan ses på figuren nedenfor. Funktionen starter med at trække pin A0 på port A LOW i 480 mikrosekunder, da dette er minimumsværdien for en reset-pulse på en DS18B20. [7]

Herefter sættes pinnen tilbage som input, så databussen frigives. Pull-up modstanden trækker da signalet HIGH igen. Sensoren venter herefter 15–60 mikrosekunder og sender et presence-pulse ved kortvarigt at trække databussen LOW for at indikere, at sensoren er tilsluttet og klar til kommunikation.

Funktionen aflæser databussen efter 70 mikrosekunder for at kontrollere, om sensoren har sendt et presence-pulse. Hvis signalet er LOW, sættes variablen response til 1, og funktionen returnerer værdien 1. Hvis databussen ikke trækkes LOW af sensoren, returneres værdien 0.




*Figur 25: Initialisering timing for DS18B20, figur 15 i databladet.*

De 4 hjælpefunktioner *writeBit*, *writeByte*, *readBit* og *readByte* er implementeret til håndtering af sensorens write- og read-time-slots. Logikken bag disse funktioner er beskrevet i afsnit 2.2 bilag D.

**readTemp():**

**Returtype:** double

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Funktionen *readTemp()* er klassens eneste public funktion, som bruges til at måle temperaturen i DS18B20 sensoren. Den returnerer temperaturen i celsius som en double.

Funktionen følger den transaktionssekvens der er angivet i databladet:

*”Step 1. Initialization*
*Step 2. ROM Command (followed by any required data exchange)*
*Step 3. DS18B20 Function Command (followed by any required data exchange)”* [7, p. 10]

Sensoren initialiseres ved at kalde reset(). Den kaldes i en if-sætning der siger, at hvis reset returnerer 0 (false) så får temperaturen værdien -1000, som fungerer som en fejlværdi, hvis sensoren ikke kommunikerer korrekt med Arduinoen.

Hvis reset() lykkes kaldes Skip ROM [CCh] efterfulgt af Convert T [44h]. Siden vi kun benytter én sensor, anvendes Skip ROM, så sensorens 64-bit ROM-kode ikke behøves. Den efterfølges af kommandoen Convert T, der fortæller sensoren den skal starte en temperaturkonvertering og gemme resultatet i scratchpad-hukommelsen.

For at læse scratchpad-hukommelsen benyttes kommandoen Read Scratchpad [BEh]. Inden den kaldes igen reset() og Skip ROM så transaktionssekvensen overholdes. Efter Read Scratchpad læses temperaturen ind på to variabler, lsb og msb. Disse samles til en 16-bit variabel raw. Temperaturen omregnes herefter til celsius ved at dividere med 16 og returneres som en double.

### 6.2.4 MaaleSession

#### 6.2.4.1 Beskrivelse af klassen:

MaaleSession er en domainklasse i systemet. Objekter af denne klasse indeholder de data, som løbende registreres og lagres under en fysisk badesession, herunder bl.a. vandforbrug, temperatur, energimængder, priser og varighed.

```cpp
#pragma once
#include <string>
#include <chrono>

class MaaleSession {
public:
    MaaleSession(int userId); //constructor svarer til initsession fra UC1 klassediagram.

    void startSession(); // start tidtagning
    void endSession(); // stop tidtagning

    int getUserId()const;
    int getShowerLength()const;
    double getTotalWaterUsed() const;
    double getAverageTemp() const;
    double getEnergyUsed() const;
    double getWaterPrice() const;
    double getEnergyPrice() const;
    double getTotalEnergyCost() const;
    double getTotalWaterCost() const;

    void setTotalWaterUsed(double volume);
    void setAverageTemp(double temp);
    void setEnergyUsed(double energy);
    void setTotalEnergyCost(double TotalEnergyCost);
    void setTotalWaterCost(double TotalWaterCost);



private:
    int Date_;
    int UserId_;
    double TotalWaterUsed_;//
    double AverageTemp_;//
    int ShowerLength_;//
    double EnergyUsed_;// omskrevet til double, håber det okay -JJ
    double WaterPrice_;//
    double EnergyPrice_;//
    double TotalEnergyCost_;//
    double TotalWaterCost_;//

    std::chrono::time_point<std::chrono::steady_clock> StartTime_;
    std::chrono::time_point<std::chrono::steady_clock> EndTime_;

};
```

*Figur 26: kodeudsnit, MaaleSession klassen*

#### 6.2.4.2 Beskrivelse af metoder:

**startSession():**

**Returtype:** void

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Metoden registrerer starttidspunktet for badesession, ved hjælp af biblioteket chrono [10], som har en indbygget clock funktion. Starttidspunktet gemmes i attributten StartTime_. Dette tidspunkt anvendes senere til at beregne varigheden af badesession.

**endSession():**

**Returtype:** void

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Metoden registrerer sluttidspunktet, som gemmes i attributten EndTime_. Derefter beregnes og gemmes den samlede badelængde i attributen Showerlength_ i sekunder ved at tage forskellen mellem start og sluttidspunktet.

### 6.2.5 MeasurementControl

#### 6.2.5.1 Beskrivelse af klassen:

MeasurementControl er en af systemets kontrol klasser, klassen står for at oprette et MaaleSession objekt, og kalde på metoder fra forskellige boundary og domain klasser i systemet undervejs i MaaleSession objektets levetid.

```
class MeasurementControl
{
public:
    MeasurementControl();
    void startSession();
    void beginMeasurement();
    void selectedUser(int userId);
    void waterLimitSet(double limit);
    void incLimit();
    void decLimit();
    void stopSession();
    void storeSession();
    void drainSensor();
    double getWaterLimit();
    int getCurrentUserId();
    int getReadingCount();
    double getTotalVolume();
    double getTempSum();

private:
    double CurrentTemp_; // seneste måling fra tempsensor og arduino
    double CurrentFlow_; // seneste måling fra flowsensor og arduino
    double TotalVolume_; // akkumuleret vandvolume
    double TempSum_;     // temp målinger lagt sammen under session
    int ReadingCount_;   // antal målinger pr session
    int CurrentUserId_;  // den valgte user
    double WaterLimit_;  // det valgte threshold ift lyd trigger.

    MaaleSession *activeSession_; // pointer til aktuelle målesession
    Calculator calculator_;       // calculator objekt, så vi kan tilgå calculator funktioner
    SensorReceiver sensorReceiver_;
    User User_;
    Database Database_;
    AccessDataControl accessDataControl_;

    std::array<User, 4> Users_{// et array af User
                               User(1, "User 1"),
                               User(2, "User 2"),
                               User(3, "User 3"),
                               User(4, "User 4")};
};
```

*Figur 27: kodeudsnit, MeasurementControl klassen*

#### 6.2.5.2 Beskrivelse af metoder:

**startSession():**

**Returtype:** void

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Starter en ny målesession for den valgte bruger. Først tjekker funktionen, om der allerede er en aktiv session, eller om der ikke er valgt en bruger endnu. Hvis alt er klar, nulstiller den de aktuelle måleværdier, så sessionen starter fra 0. Derefter oprettes en ny MaaleSession, og tidtagningen for sessionen startes.

**beginMeasurement():**

**Returtype:** void

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Metoden udfører løbende dataopsamling under den aktive badesession. Der læses aktuelle målinger fra henholdsvis Temperatur og FlowSensor klasserne, som gemmes i attributterne CurrentTemp_ og CurrentFlow_.

De aflæste værdier summes op, ved at opdatere TempSum_, TotalVolume_ og ReadingCount_, som bruges til senere beregninger. På baggrund af det de indsamlede værdier kaldes der på metoder fra Calculator klassen til løbende at beregne gennemsnittemperatur, energiforbrug og samlede omkostninger. De beregnede og akkumulerede værdier skrives efterfølgende til en fil, som systemets GUI bruger til at vise data live for brugeren.

**stopSession():**

**Returtype:** void

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Metoden afslutter badesessionen ved at stoppe for tidtagningen og beregner den samlede badelængde, ved at kalde på endSession()-metoden fra MaaleSession. Derefter kaldes der på metoderne fra Calculator klassen for at beregne værdier der efterfølgende skal gemmes i MaaleSession objektet.

Når værdierne er beregnet og gemt kaldes metoden storeSession, som gemmer MaaleSession objektet i Databasen, hvorefter pointeren sættes til nullptr og klassens attributer nulstilles så systemet er klar til en ny badesession.

### 6.2.6 GUI

#### 6.2.6.1 Beskrivelse af klassen

GUI'en er systemets boundary klasse mod brugeren. Klassen er implementeret som en Flask-webapplikation og står for at modtage brugerens input, videresende kommandoer til C++ systemet samt hente og vise måledata og statistik til brugeren.

Til dette projekt har vi valgt at bruge Flask til at lave vores grafiske brugergrænseflade. Flask er et simpelt webudviklingsframework til Python som passer godt til prototyper. Da vi ikke tidligere har arbejdet med grafiske brugergrænseflader, valgte vi at starte på begynderniveau frem for at kaste os ud i noget mere komplekst. Flask gør det muligt at køre en webserver direkte på Raspberry Pi'en så systemet kan tilgås fra en browser på samme netværk uden at kræve installation af software på brugerens enhed.

```python
from flask import Flask, render_template, request, redirect
import os

app = Flask(__name__) # opretter webapplikationen

# projektets rodmappe – en mappe op fra GUI/
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))

# konstanter er navnene på de tekstfiler som Flask og C++ deler med hinanden
COMMAND_FILE = os.path.join(BASE_DIR, "command.txt")
STATS_FILE   = os.path.join(BASE_DIR, "session_data.txt")
LIVE_FILE    = os.path.join(BASE_DIR, "live_data.txt")
```

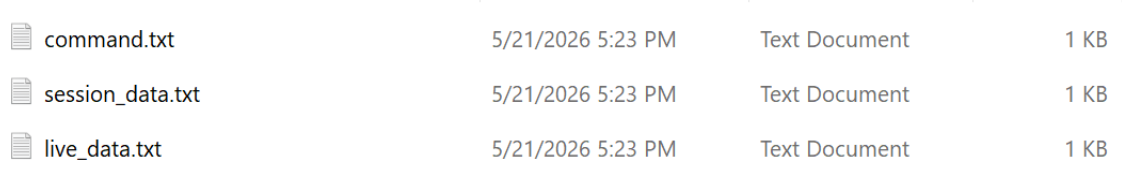
*Figur 28- Kodeudsnit, app.py header*

Vores GUI er bygget op af fire HTML-filer som udgør layoutet på vores forskellige sider: start.html, settings.html, liveData.html og statistics.html. HTML-filerne indeholder næsten ingen logik, de er kun til at formatere siderne og give brugeren mulighed for at navigere rundt i systemet. Selve logikken ligger i Flask-applikationen app.py som er hjernen i systemet og binder alle dele sammen.




*Figur 29 - GUI Mappe struktur*

Da Flask og C++-programmet er to separate processer der kører uafhængigt af hinanden på Raspberry Pi'en kan de ikke kommunikere direkte. I stedet deles data via tre tekstfiler som Flask enten skriver til eller læser fra afhængigt af filens formål.




*Figur 30 - txt filer oprettes efter programmet køres*

Live_data.txt er den fil MeasurementControl klassen i C++ skriver løbende måledata til under en aktiv session med vandmængde, temperatur, energi og pris. Flask læser denne fil og viser de opdaterede værdier på liveData.html. For at undgå at Flask læser filen midt i en skrivning og får ufuldstændige data skriver C++ først til en midlertidig fil live_data.tmp som derefter omdøbes til live_data.txt. På den måde ser Flask altid enten den gamle eller den nye version, aldrig noget midt imellem.

session_data.txt indeholder akkumulerede sessionsdata per bruger. Filen skrives af accessDataControl_ via updateLeaderboard(), der kaldes fra storeSession() i MeasurementControl. Flask læser filen og viser dataene sorteret på statistics.html.

command.txt fungerer omvendt af de to andre filer: her er det Flask der skriver og C++ main-loopet der læser og kalder de relevante metoder i MeasurementControl.

app.py definerer tre hjælpefunktioner til filkommunikationen samt syv URL-routes der håndterer de HTTP-forespørgsler brugeren genererer ved at navigere i systemet.

#### 6.2.6.2 Beskrivelse af metoder

**write_command():**

**Returtype:** Ingen

**Parametre:** cmd - en tekststreng indeholdende den kommando der skal skrives til command.txt, eksempelvis START:2:15

**Beskrivelse af funktionalitet:** Metoden fungerer som kommunikationsbroen mellem Flask-applikationen og C++ systemet. Når brugeren eksempelvis starter eller stopper et bad, skriver metoden en kommandostreng til filen command.txt, som C++ main-loopet løbende poller. Eksempelvis skrives strengen START:2:15 til filen, når bruger 2 starter et bad med et vandinterval på 15 liter. Denne tilgang er valgt, da Flask og C++ kører som separate processer og ikke kan kommunikere direkte.

**read_statistics():**

**Returtype:** En liste med data for hver bruger, sorteret efter energi.

**Parametre:** Ingen

Metoden læser sessiondata fra filen session_data.txt, som C++ skriver til når en badesession afsluttes. Filen indeholder én linje pr. bruger med det samlede vandforbrug, temperatur, energi og pris på tværs af alle tidligere sessioner. Disse værdier aflæses og bruges direkte til at vise

leaderboardet, hvor brugerne sorteres stigende efter energiforbrug, så den mest energibesparende bruger vises øverst. Hvis filen endnu ikke eksisterer, vises alle brugere med værdier på 0, så applikationen ikke fejler.

### 6.2.7 Sound

#### 6.2.7.1 Beskrivelse af klassen:

Sound er en boundary-klasse, der "pakker" Somo-klassen, så resten af systemet ikke skal forholde sig til UART-kommandoer direkte. Klassens primære ansvarsområde er at gøre kommunikationen med SOMO-II lydmodulet enklere for resten af systemet, ved at samle valg af lydfil, lydstyrke og start/stop af afspilning i et sted. Sound holder selv styr på hvilken track der skal afspilles, den aktuelle lydstyrke, samt om modulet er blevet initialiseret, så de øvrige klasser i lyd-pakken (primært AfspilLyd) ikke skal håndtere de UART-kommandoer direkte.

```
#pragma once
#include "SOMO.h"
#include <stdint.h>
class Sound {
public:
    Sound(uint8_t trackNumber = TRACK_INTERVAL, uint8_t volume = 25);

    void begin();
    bool playSound();
    void stopSound();
    void setVolume(uint8_t volume);
    void setSoundFile(uint8_t trackNumber);

private:
    Somo somo_;
    uint8_t folderNumber_= SOMO_FOLDER_01;
    uint8_t trackNumber_;
    uint8_t volume_;
    bool initialized_;
};
```

*Figur 31: kodeudsnit Sound klassen i Sound.h*

#### 6.2.7.2 Beskrivelse af metoder:

**playSound()**

**Returtype:** bool

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Metoden afspiller den valgte mp3-fil på SOMO-II modulet. Først tjekkes attributten initialized_, og hvis modulet ikke er initialiseret endnu, kaldes begin() automatisk. Dette er en form for sikring, så afspilning altid virker. Herefter kaldes

somo_.playFolderTrack() med folderNumber_ og trackNumber_, som sender en UART-kommando til SOMO-II om at afspille den valgte mappe og track. Metoden returnerer true for at signalere, at afspilningskommandoen er sendt.

**setSoundFile(uint8_t trackNumber)**

**Returtype:** void

**Parametre:** uint8_t trackNumber

**Beskrivelse af funktionalitet:** Metoden vælger hvilken mp3-fil der skal afspilles ved næste kald til playSound(). Den nye værdi gemmes blot i attributten trackNumber_, og selve afspilningen sker først når playSound() kaldes. Dette gør det muligt for AfspilLyd at skifte mellem fx TRACK_START, TRACK_INTERVAL og TRACK_SLUT alt efter hvilken tilstand sessionen er i.

### 6.2.8 AfspilLyd

#### 6.2.8.1 Beskrivelse af klassen:

AfspilLyd er en control-klasse, der styrer logikken for hvornår der skal afspilles lyd under en aktiv badesession. Klassen holder løbende øje med det samlede vandforbrug og afspiller en lyd hver gang et nyt litergrænseinterval nås, fx ved 5, 10, 15 liter osv. Selve afspilningen håndteres ikke af AfspilLyd selv, men uddeles til et Sound-objekt, som klassen modtager en reference til via constructoren. AfspilLyd kobles dermed sammen med lyd-pakken (det der består af AfspilLyd, Sound og Somo). uden at kende til UART-kommandoerne i SOMO-II modulet, da det er SOMO, der ved, hvordan det skal sendes (UART-bytes og kommandokoder fra databladet).

```
#pragma once
#include "Sound.h"

class AfspilLyd {
public:

    AfspilLyd(Sound& sound, double waterLimitLiters = 5.0);

    void begin();
    void reset();
    void setWaterLimit(double waterLimitLiters);
    void setEnabled(bool enabled);

    bool checkAndPlay(double totalVolumeLiters, bool waterFlowActive);
    double getNextThreshold() const;

private:
    Sound& sound_;
    double waterLimitLiters_;
    double nextThresholdLiters_;
    bool enabled_;
};
```

*Figur 32: kodeudsnit AfspilLyd klassen i AfspilLyd.h*

#### 6.2.8.2 Beskrivelse af metoder:

**setWaterLimit(double waterLimitLiters)**

**Returtype:** void

**Parametre:** double waterLimitLiters

**Beskrivelse af funktionalitet:** Metoden ændrer hvor ofte lyden skal afspilles ved at sætte en ny værdi for waterLimitLiters_. Først tjekkes at den nye værdi er gyldig, dvs. større end 0, så der ikke kan sættes en ugyldig grænse. Hvis værdien er gyldig, gemmes den og reset() kaldes, så næste lydgrænse passer til den nye indstilling. Metoden bruges fra main(), når brugeren via GUI'en har valgt et nyt vandinterval i forbindelse med start af en session.

**checkAndPlay(double totalVolumeLiters, bool waterFlowActive)**

**Returtype:** bool

**Parametre:** double totalVolumeLiters, bool waterFlowActive

**Beskrivelse af funktionalitet:** Metoden udgør AfspilLyd-klassens kerne og kaldes løbende fra main() hver gang nye flowdata er målt under en aktiv session. Først tjekkes om lyden er slået til (enabled_), om der reelt løber vand (waterFlowActive), og om vandgrænsen er gyldig. Hvis ikke, returneres false uden at afspille lyd. Hvis det samlede vandforbrug (totalVolumeLiters) har nået eller overskredet nextThresholdLiters_, sættes den korrekte lydfil via sound_.setSoundFile(TRACK_INTERVAL), hvorefter sound_.playSound() afspiller lyden. Herefter flyttes nextThresholdLiters_ frem i et while-loop, så længe vandforbruget stadig er over den næste grænse. Dette håndterer det tilfælde, hvor der mellem to målinger er løbet mere end et interval igennem, så lyden ikke fejlagtigt afspilles flere gange i træk. Metoden returnerer true, hvis der blev afspillet en lyd, og ellers false.

```
//den kaldes fra MeasurementControl::beginMeasurement() hver gsng ny flowdata er målt
//eksempel: afspilLyd.checkAndPlay(TotalVolume_, CurrentFlow_ > 0.0);
//returnrer true, hvis den afspillede en lyd. Returnerer false, hvis den ikke gjorde
bool AfspilLyd::checkAndPlay(double totalVolumeLiters, bool waterFlowActive) {
    if (!enabled_ || !waterFlowActive || waterLimitLiters_ <= 0.0) {              //stop hvis slyd er slukket, der ikke er flow, eller grønsen er ugyldig
        return false;                                                             //hvisn ingen lyd blev afspillt
    }

    if (totalVolumeLiters >= nextThresholdLiters_) {              //hvis samlet vandforbrug har nået næste grænse...
        sound_.setSoundFile(TRACK_INTERVAL);
        sound_.playSound();                                       //...så afspilles den valgte lyd via Sound-klassen

        //sørge for, at næste lyd kommer ved næste grænse, fx 5, 10... liter
        while (totalVolumeLiters >= nextThresholdLiters_) {       //hvis målingen sprang over flere grænse...
            nextThresholdLiters_ += waterLimitLiters_;      //...flyttes næste grænse frem, indtil den lægger over totalVolumeLiters
        }

        return true;                                      //fortælle rresten af programmet at en lyd blev afspillelt
    }

    return false;                                         //hvis grænsen ikke er nået endnu, afspilles ingen lyd
}
```

*Figur 33: kodeudsnit checkAndPlay metoden i AfspilLyd.cpp*

### 6.2.9 Somo

#### 6.2.9.1 Beskrivelse af klassen:

Somo er en driver-klasse, der håndterer den direkte kommunikation med SOMO-II lydmodulet via UART1 på Arduinoen (ATmega2560). Klassen er en del af "Somo_main"-pakken, som indeholder filerne SOMO.h, SOMO.cpp, Sound.h, Sound.cpp og SOMO_main.ino, og som tilsammen udgør programmet for Arduino enheden, der fungerer som Sound Controller i systemet. Denne Arduino er separat fra Sensor Controller Arduinoen og kører sit eget program via Arduino IDE 2.0. Klassens primære ansvarsområde er at omsætte simple metodekald som playFolderTrack() og setVolume() til de korrekte 8-byte kommandoer som SOMO-II's datablad beskriver, og sende dem ud på TX1-pinnen som bytes. Klassen indkapsler dermed både opsætningen af UART hardwaren, selve kommandoformatet (start-byte, kommando, parametre, checksum og slut-byte) og beregningen af checksum, så Sound-klassen ikke skal forholde sig til disse.

```
#pragma once
#include <stdint.h>

class Somo
{
private:
    static const uint8_t START_BYTE = 0x7E;                    //start-byte for alle SO
    static const uint8_t END_BYTE   = 0xEF;                    //slut-byte for all SOMO

    static const uint8_t FEEDBACK_OFF = 0x00;                  //0 betyder, at SOMO-II

    static const uint8_t CMD_PLAY_TRACK_SECONDARY = 0x03;      //afspiller track efter
    static const uint8_t CMD_SET_VOLUME           = 0x06;      //sætter lydstyrken mell
    static const uint8_t CMD_PLAY                 = 0x0D;      //starter eller fortsætt
    static const uint8_t CMD_PAUSE                = 0x0E;      //pauser afspilning
    static const uint8_t CMD_PLAY_FOLDER_TRACK    = 0x0F;      //afspiller mappe + trac
    static const uint8_t CMD_STOP                 = 0x16;      //stopper afspilning

    static const uint8_t VOLUME_MAX = 30;       //maksimal lydstyrke på SOMO-II
    static const uint8_t VOLUME_MIN = 0;       //minimal lydstyrke på SOMO-II
    static const uint8_t VOLUME_DEFAULT = 25;

    unsigned long baudrate_;

    void uartInit();
    void uartSend(uint8_t byte);
    void sendCommand(uint8_t cmd, uint8_t param1, uint8_t param2);  //sender hel SOMO-II

public:
    Somo(unsigned long baudrate = 9600UL);

    void begin();
    void playFolderTrack(uint8_t folderNumber, uint8_t trackNumber);    //spiller efter
    void setVolume(uint8_t volume);
    void stop();
    void pause();
    void play();
};

static const uint8_t SOMO_FOLDER_01 = 0x01;
static const uint8_t TRACK_START    = 0x01;
static const uint8_t TRACK_INTERVAL  = 0x02;
static const uint8_t TRACK_SLUT     = 0x03;
```

*Figur 34: kodeudsnit Somo klassen i SOMO.h*

#### 6.2.9.2 Beskrivelse af metoder:

**playFolderTrack(uint8_t folderNumber, uint8_t trackNumber)**

**Returtype:** void

**Parametre:** uint8_t folderNumber, uint8_t trackNumber

**Beskrivelse af funktionalitet:** Metoden afspiller en specifik mp3 fil ved at angive mappenummer og tracknummer på SD-kortet i SOMO-II modulet. Den kalder den private hjælpefunktion sendCommand() med kommandoen CMD_PLAY_FOLDER_TRACK (0x0F), og parametrene folderNumber og trackNumber. Et eksempel kan være folderNumber = 0x01 og trackNumber = 0x02, som afspiller filen 002xxx.mp3 i mappen 01 på SD-kortet.

```
void Somo::playFolderTrack(uint8_t folderNumber, uint8_t trackNumber)
{
    //dene funktion bruge kommando 0x0F som er "specify folder &t rack"
    //eksempel: folderNumber = 0x01 og trackNumber = 0x02 spiller fil 002xxx.mp3 i mappen 01
    sendCommand(CMD_PLAY_FOLDER_TRACK, folderNumber, trackNumber);
}
```

*Figur 35: kodeudsnit playFolderTrack metoden i SOMO.cpp*

Udover disse public metoder indeholder klassen tre private hjælpefunktioner: uartInit(), uartSend(byte) og sendCommand(cmd, param1, param2), som tilsammen håndterer opsætning af UART1, transmission af enkelte bytes og opbygning af den fulde 8-byte kommandopakke. Særligt sendCommand() er central, da den beregner checksum efter formlen fra databladet: checksum = 0xFFFF - (CMD + Feedback + Param1 + Param2) + 1, og sender de 8 bytes i den korrekte rækkefølge: START_BYTE, CMD, FEEDBACK_OFF, param1, param2, checksum high, checksum low, END_BYTE.

```
void Somo::sendCommand(uint8_t cmd, uint8_t param1, uint8_t param2)
{
    //SOMO kommandoformatet er 8 byte: START, CMD, FEEDBACK, PARAM1, PARAM2, CHECKSUM_HIGH, CHECKSUM_LOW, END

    uint16_t sum = static_cast<uint16_t>(cmd) + FEEDBACK_OFF + param1 + param2;
    //sum er summen af de fire bytes, som indgår i checksum beregningne


    uint16_t checksum = 0xFFFF - sum + 1;   //ligger kommando dele sammen checksum formeln følger SOMO datablade

    uartSend(START_BYTE);                           //byte 1: start på kommandoen, altid 0x7E
    uartSend(cmd);                                  //byte 2: selve kommandoen, fx 0x0F for mappe+ track
    uartSend(FEEDBACK_OFF);                         //byte 3: feedback slået fra
    uartSend(param1);                               //byte 4: parameter 1, fx mappe nummer
    uartSend(param2);                               //byte 5: parameter 2, fx track nummer
    uartSend(static_cast<uint8_t>(checksum >> 8));     //byte 6: checksum high byte
    uartSend(static_cast<uint8_t>(checksum & 0xFF));   //byte 7: checksum low byte
    uartSend(END_BYTE);                             //byte 8: slut på kommandoe n altid 0xEF

#if defined(__AVR__)
    _delay_ms(20);
#endif
}
```

*Figur 36: kodeudsnit sendComand metoden i SOMO.cpp*

### 6.2.10 AccessDataControl

#### 6.2.10.1 Beskrivelse af klassen:

AccessDataControl er en af systemets subkontrol klasser. Klassen står for at aggregere de MaaleSession objekter, som Database klassen har gemt under tidligere badesessioner, og skrive det samlede resultat ud i den form GUI'en skal bruge. Klassen udfører ingen beregninger på den enkelte session, men summerer data på tværs af alle sessions for at danne en samlet rangliste over husstandens brugere. Klassen anvendes i forbindelse med UC 3: Tilgå data. AccessDataControl holder en reference til systemets Database objekt, som modtages i konstruktøren og gemmes i attributten database_. Klassen ejer ikke databasen, men tilgår blot de sessions den indeholder. Når en bruger afslutter en badesession, kaldes MeasurementControl::storeSession(), som gemmer sessionen i Database via Database::saveSession() og derefter kalder AccessDataControl klassens updateLeaderboard(), som summerer og opdaterer alle gemte sessions pr. bruger og skriver det aggregerede resultat til tekstfilen session_data.txt. Flask læser herefter filen og viser ranglisten på statistiksiden, hvor brugerne sorteres efter samlet energiforbrug.

```
#pragma once

#include "Database.h"
#include "Maalesession.h"
#include <vector>

class AccessDataControl
{
public:
    AccessDataControl(Database &database);
    void updateLeaderboard();

private:
    Database &database_;
};
```

*Figur 37: kodeudsnit, AccessDataControl klassen i AccessDataControl.h*

#### 6.2.10.2 Beskrivelse af metoder:

**AccessDataControl(Database& database)**

**Returtype:** ingen (konstruktør)

**Parametre:** Database& database

**Beskrivelse af funktionalitet:** Konstruktøren modtager en reference til systemets Database objekt og gemmer den i medlemsvariablen database_. Da der anvendes en reference, deler

AccessDataControl samme Database objekt som resten af systemet og arbejder dermed altid på de aktuelt gemte sessions

**updateLeaderboard():**

**Returtype:** void

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Metoden producerer den samlede rangliste over husstandens brugere og skriver den til tekstfilen session_data.txt, som Flask efterfølgende læser fra. Først hentes alle gemte sessions fra Database via getSessions(). Derefter løbes listen igennem, og for hver session lægges sessionens TotalWaterUsed, AverageTemp, samlede pris (TotalWaterCost + TotalEnergyCost) og EnergyUsed sammen pr. userId. Til formålet bruges fem std::map containere, hvor userId fungerer som nøgle, så hver brugers værdier summeres på tværs af deres sessions. Antallet af sessions pr. bruger tælles i countPerUser, så gennemsnitstemperaturen kan beregnes som tempSum delt med antallet af sessions. Når alle sessions er løbet igennem, åbnes session_data.txt, og hver bruger skrives som en linje på følgende vis: userId, totalWater, averageTemp, totalCost, energy, adskilt af kommaer. Der itereres over waterPerUser for at tilgå nøglerne (userId), men en hvilken som helst af de øvrige maps kunne være anvendt, da de deler samme nøgler. Sorteringen af ranglisten efter pris foretages i Flask, da Flask alligevel skal læse og parse filen. Hver gang metoden kaldes, overskrives session_data.txt med de nyeste aggregerede tal.

```
void AccessDataControl::updateLeaderboard()
{
    vector<MaaleSession> sessions = database_.getSessions();

    // summer data per bruger (1,2,3,4)
    map<int, double> waterPerUser;
    map<int, double> tempSumPerUser;
    map<int, double> costPerUser;
    map<int, double> energyPerUser;
    map<int, int> countPerUser;

    for (const auto &s : sessions)
    {
        int id = s.getUserId();                    //Finder user ID for hvert enkelt sessio
        waterPerUser[id] += s.getTotalWaterUsed();
        tempSumPerUser[id] += s.getAverageTemp();
        costPerUser[id] += s.getTotalWaterCost() + s.getTotalEnergyCost();
        energyPerUser[id] += s.getEnergyUsed();
        countPerUser[id] += 1;
    }

    ofstream file(STATS_FILE);
    for (const auto &element : waterPerUser)       //Vi anvender waterPerUser men kunne lig
    {
        int id = element.first; // Her kan vi tilgå nøglen (user id)
        double water = element.second;
        double temp = tempSumPerUser[id] / countPerUser[id];
        double cost = costPerUser[id];
        double energy = energyPerUser[id];
        file << id << "," << water << "," << temp << "," << cost << "," << energy << "\n";
    }
}
```

*Figur 38: kodeudsnit, updateLeaderboard() metoden fra AccessDataControl klassen*

### 6.2.11 Database

#### 6.2.11.1 Beskrivelse af klassen:

Database er en domainklasse, der fungerer som systemets interne lager for de gennemførte målesessioner. Klassen holder samtlige MaaleSession objekter i en privat std::vector og udstiller funktionalitet til at gemme nye sessioner samt hente dem ud igen samlet. På denne måde adskilles datalagringen fra kontrol laget, så øvrige klasser ikke behøver kende til den underliggende opbevaring. Database ejes af MeasurementControl, der via sin reference videregiver objektet til AccessDataControl, så begge klasser arbejder på samme samling af sessions.

```
#pragma once

#include "Maalesession.h"
#include <vector>

class Database {
public:
    Database();

    void saveSession(const MaaleSession& session);

    std::vector<MaaleSession> getSessions() const;


private:
    std::vector<MaaleSession> sessions_;
};
```

*Figur 39: kodeudsnit, Database klassen i Databse.h*

#### 6.2.11.2 Beskrivelse af metoder:

**Database()**

**Returtype:** ingen

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Standardkonstruktøren initialiserer et tomt Database objekt. Den private vector sessions_ oprettes automatisk tom, så databasen ved opstart ikke indeholder nogen sessions.

**saveSession(const MaaleSession& session)**

**Returtype:** void

**Parametre:** const MaaleSession& session

**Beskrivelse af funktionalitet:** Metoden tilføjer en afsluttet målesession til databasen. Det modtagne MaaleSession objekt lægges bagerst i den private vector sessions_ via push_back(). Parameteren modtages som en konstant reference for at undgå unødig kopiering af objektet og samtidig sikre, at den oprindelige session ikke ændres.

**getSessions()**

**Returtype:** std::vector<MaaleSession>

**Parametre:** ingen

**Beskrivelse af funktionalitet:** Metoden returnerer samtlige gemte sessioner som en vector. Den er deklareret const, da den ikke ændrer databasens tilstand, men blot leverer en kopi af listen. Metoden benyttes, hvor der er behov for samtlige registrerede data, eksempelvis når AccessDataControl opbygger leaderboardet på tværs af brugere.

# 7. Integration og test

## 7.1 Integrationstest

| Test: | Beskrivelse: | Problem: | Løsning: | Resultat: |
|---|---|---|---|---|
| 1. | Software overføres til Arduinoer og Raspberry Pi. | Microchip Studio understøtter ikke Arduino-frameworket. Funktioner som Serial.begin() kunne ikke kompileres. | Skift til Arduino IDE 2, som har Arduino-frameworket indbygget. | Koden kompileres og overføres uden problemer |
| 2. | Flowsensor testes | Ingen bemærkelsesværdige problemer. | - | Der forekommer et mindre delay af værdier, men på daværende tidspunkt vurderes det ikke til at være et reelt problem |
| 3. | Temperatursensor testes sammen med flowsensor | Ingen bemærkelsesværdige problemer. | - | Temperaturværdier vises på GUI, og værdier sammenlignes med eksternt termometer. Ens resultat på termometer og GUI. |
| 4. | SOMO testes sammen med temperatur og flowsensor | SOMO og sensorer fungerer ikke optimalt, SOMO afspiller ikke lyd ved forventede tidspunkter. | SOMO flyttes til separat Arduino | Sensorer og SOMO fungerer korrekt hver for sig. |

| | | | | |
|---|---|---|---|---|
| 5. | Gentagende forsøg med vand igennem systemet | GUI viser ca. 0.5 liter for meget ift. den målte vægt på køkkenvægt. | Kalibrering af flowsensor justeres | Måling forbedres, men mindre afvigelser forekommer stadig. |
| 6. | System testes igen efter kalibrering | GUI er langsom til at vise live data af samlet vandmængde. Der forekommer latenstid, som varierer i længde. | Analyse af flowsensorens målemetode | GUI opdateres langsomt i starten, men bliver hurtigere over tid og til sidst er synkron med målt værdier på vægt. |
| 7. | Efter kalibrering testes systemet igen | GUI opdateres stadig langsomt i starten, men bliver hurtigere over tid og til sidst er synkron med målt værdier på vægt. | Der udarbejdes en drain metode der håndterer ophobning af data i den serielle kommunikation fra flowsensoren. | - |
| 8. | Efter implementering af drain metode i Sensor Receiver klassen testes system. | Der forekommer ingen problemer med den serielle kommunikation længere. Dataene som vises på GUI, vises synkront med den målte mængde på vægten. | Drain metode fungerer, og der er derfor ikke behov for flere justeringer på nuværende tidspunkt | Systemet fungerer, ud fra de nuværende krav. |

*Tabel 3: Integrationstest*

# 8. Accepttest

## 8.1 Prækondition for accepttest

**Tilkobling af system:**

1. Vanddunken fyldes med 10 liter vand og placeres på et bord.
2. Temperatursensoren placeres i vanddunken, så måleområdet er dækket af vand.
3. Køkkenvægten placeres på gulvet i nærheden af vanddunken.
4. Kanden placeres ovenpå vægten.
5. Vægten nulstilles og indstilles til gram.
6. Plastikrøret fra flowsensorens udgang placeres i kanden.
7. Arduinoerne forbindes til Raspberry Pi via USB.
8. HDMI- og USB-kabler tilsluttes mellem Raspberry Pi og touchskærm.
9. Sensorerne tilkobles én Arduino via VERO-board.
10. SOMO-modul og højttaler tilkobles den anden Arduino.

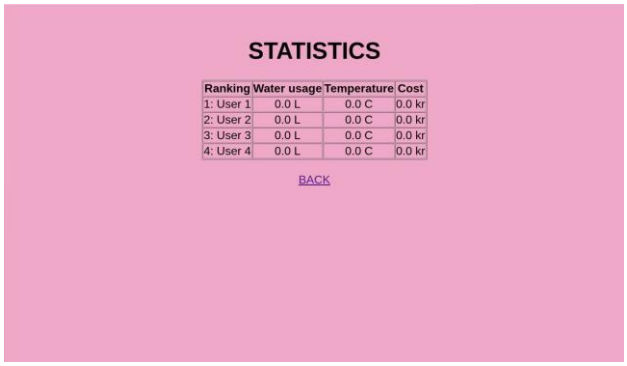
**Opstart af system:**

1. Raspberry Pi og touchskærm tilsluttes strømforsyning.
2. Enhederne tændes, som følge af tilslutning.
3. Touchskærmen viser skrivebordet.
4. En terminal åbnes.
5. Kommandoen ./start.sh indtastes og køres.
6. Start.sh bash script starter Backend (C++).
7. Scriptet starter dernæst GUI (app.py).
8. Scriptet åbner en Chromium-browser med systemhjemmesiden i kiosk mode (fuldskærm).
9. Hovedmenuen vises – systemet er nu klar til brug.

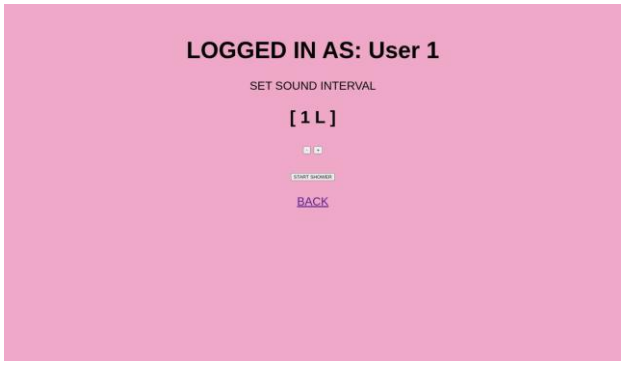



*Figur 40: GUI hovedmenu*

Hovedmenu vises med mulighed for at vælge 1-4 users eller statistik funktion, som viser brugernes forbrug efter endt badesession.

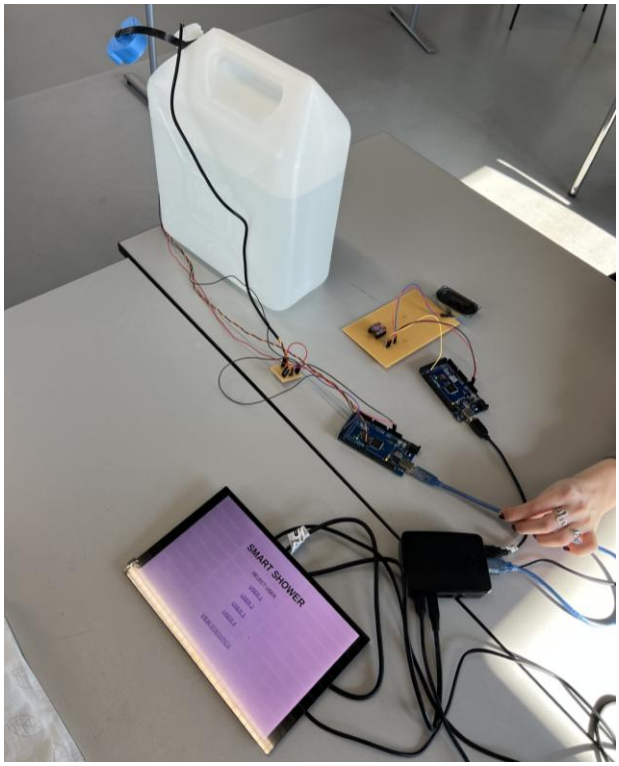



*Figur 41: GUI – statistics*

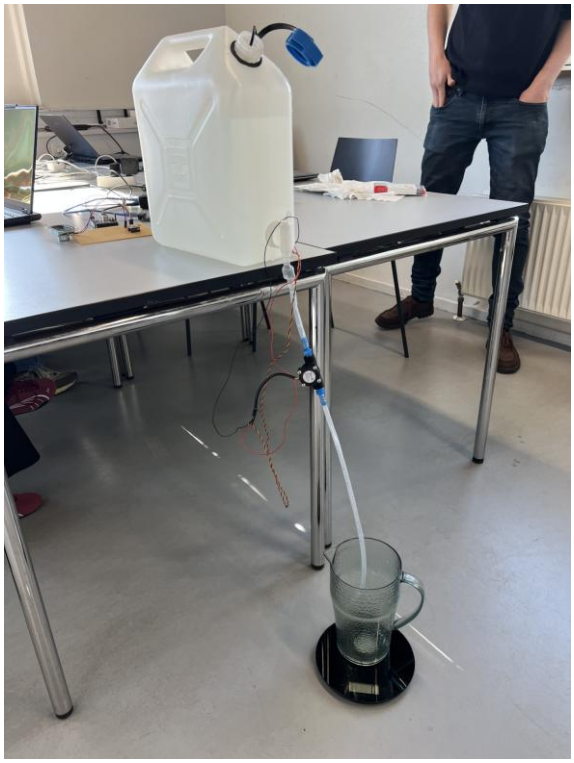



*Figur 42: GUI – user 1 er valgt, og session kan startes*

Det er en forudsætning at Raspberry Pi har loadet den seneste kode fra den fælles gitlab server, og at filerne er tilgængelig på Raspberry Pi.




*Figur 43:Prototype setup*



*Figur 44: Prototype setup*

## 8.2 Accepttest funktionelle krav

| Krav (ID & beskrivelse) | Test (ID / Navn) | Hvordan (Udførelse af test) | Forventet resultat | Resultat |
|---|---|---|---|---|
| **FR-01, FR-02** Systemet skal måle vandmængde og temperatur | **1: Sensor-måling** | Prækondition: GUI er startet og hovedmenu vises.<br>En af de 4 brugere vælges på skærmen, ved at trykke knap tilhørende den user der ønskes at tages i brug. Dernæst loades et nyt vindue og der trykkes på touchskærmens start shower knap.<br>Tænd vandet og lad 1 liter vand løbe igennem systemet.<br>Vandet samles i en kande. Når der er passeret 1 liter igennem systemet, slukkes der for vandet, og der trykkes end shower på touchskærm.<br>Vand målt i kande sammenlignes med den viste værdi på skærm.<br>Temperaturen måles med termometer.<br>Sensor målinger og egne målinger sammenlignes. Er de ens? | Vandmængde og temperatur vises korrekt (±2°C og ±5% accepteret afvigelse). | Vi har sendt 1,02 L igennem men reelt er der 1,2L og temperatur er der afvigelse på 0,2 grader.<br>Ikke godkendt. |
| **FR-03, FR-07** Systemet skal beregne energiforbrug og vise | **2: Realtids-beregning** | Prækondition: Bruger og waterlimit valgt.<br>Tryk på touchskærmens start shower knap. | Energiforbrug beregnes korrekt og opdateres løbende på skærmen (afvigelse på ±0,1 kWh godkendes) | Ikke godkendt |

| | | | | |
|---|---|---|---|---|
| data i realtid | | Lad vandet løbe i 30 sekunder. Og derefter sluk for taphane og tryk på end shower knap.<br>Observer energiforbrug beregning er korrekt, ved at efterberegne den via noteret formel. | **Formler findes under tabellen. *** | |
| **FR-04** Systemet skal afspille lyd ved forudbestemt vandmængde | **3: Lydsignal** | Prækondition: Bruger og waterlimit valgt.<br>Tryk på touchskærmens start shower knap.<br>Indstil grænse til 1 liter.<br>Lad 1 liter (kig på touchskærm) vand løbe igennem system og ned i kanden.<br>Sluk vandet når lydsignalet starter. | Lyd afspilles præcist når grænsen nås (afvigelse på ±5% godkendes) | Godkendt |
| **FR-05, FR-06** Systemet skal understøtte login og vise rangliste | **4: Bruger og rangliste** | Prækondition: GUI er genstartet.<br>To brugere gennemfører hver en session. Gå tilbage til hovedmenu og vælg view statistics og observer målinger er gemt korrekt. | Data gemmes korrekt på hver bruger. Ranglisten viser korrekt placering. | Godkendt |
| **FR-08, FR-01** Systemet skal kunne starte/stoppe session og kun registrere ved vandflow | **5: Session-kontrol** | Prækondition: Bruger og waterlimit valgt.<br>Tryk på touchskærmens start shower knap.<br>Vent 10 sekunder med at starte vandet.<br>Tænd vand og lad det løbe i 10 sekunder, og sluk vandet. | Ingen registrering uden vandflow. Session starter og afsluttes korrekt. | Betinget godkendt - Forsinkelse skal undersøges |

| | | | | |
|---|---|---|---|---|
| | | Vent 5 sekunder og afslut måling. | | |

*Tabel 4: Accepttest for funktionelle krav*

***Formler til Test 2:**

$$\boldsymbol{KWh} = \frac{TotalVolume \cdot 4186 \cdot (AverageTemp - 8)}{3600000}$$

$$\boldsymbol{KWh} = \frac{SamletPris - Liter \cdot 0{,}065}{2{,}5}$$

## 8.3 Accepttest non-funktionelle krav

| **Krav (ID & beskrivelse)** | **Test (ID / Navn)** | **Hvordan (Udførelse af test)** | **Forventet resultat** | **Resultat** |
|---|---|---|---|---|
| **NFR-1.1, NFR-1.2** Lyd skal kunne høres tydeligt og afspilles uden forsinkelse | **1: Lydtest** | Prækondition: session er i igangsat af bruger.<br>Tænd for taphanen.<br>Registrer mængde måling på ”skærm”.<br>Ved waterlimit grænse observeres lydsignal. | Lyd kan tydeligt høres på 1 meters afstand. Lyd afspilles uden mærkbar forsinkelse. | Godkendt i anden forsøg |
| **NFR-3.1, NFR-3.2:** Intuitiv navigation og tydelig menu placering | **2: Navigationstest** | Prækondition: GUI er aktiveret og hovedmenu vises på skærm.<br>Bruger klikker på menupunkt user 1-4 knap eller | Maksimalt ét klik kræves. | Godkendt |

| | | | | |
|---|---|---|---|---|
| | | view statistics knap.<br>Skærmen opdateres med undermenu<br>Der observer antal klik og menu indikator. | | |
| **NFR-3.3:** Data skal være letlæselige under bad | **3: Læselighedstest** | Prækondition: Touchskærmen viser resultat fra måling.<br>Observer skærm fra ca. 1–2 meters afstand under aktiv session. | Pris, liter og temperatur kan aflæses tydeligt og forstås hurtigt. | Godkendt |
| **NFR-2.3** Systemet skal være vandtæt ved maksimalt vandtryk | **4: Vandtæthedstest** | Tænd for taphanen ved maksimalt vandtryk.<br>Efter 30 sekunder slukkes der for taphanen | Ingen lækage eller funktionssvigt forekommer. | Godkendt |
| **NFR-2.1, NFR-2.2** Måleusikkerhed < 5% og temperaturrespons ≤ 5 sek. | **5: Præcision og responstid** | Prækondition: Bruger og waterlimit valgt.<br>Bruger åbner for taphanen og vand løber igennem systemet.<br>Der måles en start temperatur. På skærm og eksternt termometer | Vandmåling har < 5% afvigelse. Temperatur opdateres inden for 5 sekunder. | Vandmåling fejlet<br>Temperatur godkendt |

| | | | | |
|---|---|---|---|---|
| | | Bruger justerer på vand varme.<br><br>Stopur startes og responstiden observeres.<br><br>Vand temperatur måles med eksternt termometer og der observeres afvigelse mellem registreringer. | | |
| **NFR-4.1** konfigurerbar grænseværdi | **6: Support og konfiguration** | Bruger vælger imellem User 1-4.<br><br>Vinduet opdateres og bruger ændrer grænseværdi for lydsignal i systemets indstillinger.<br><br>Tænd vand, afvent vandgrænse på GUI og vægt nåes, og registrer lyd ved grænse. | Lyd registreres ved ændret vandgrænse | Godkendt |

*Tabel 5: Accepttest for ikke-funktionelle krav*

# 9. Diskussion

**Softwarearkitektur og kommunikation:**

I forbindelse med integrationstesten oplevede vi flere uforudsete problemer ift. systemets softwarearkitektur og kommunikationen mellem de forskellige hardwareenheder. Dette medførte, at vi undervejs måtte ændre flere af vores oprindelige designvalg og udvikle nye løsninger for at få systemet til at fungere stabilt. Gennem arbejdet fik vi samtidig større forståelse for de udfordringer, der kan opstå i embeddede systemer, særligt ift. kommunikation mellem forskellige typer hardware og håndtering af live data.

Et af de første problemer vi stødte på var, at Microchip Studio ikke understøttede det Arduino-specifikke framework, som vores kode gjorde brug af. Dette kom bl.a. til udtryk ved brugen af funktioner som Serial.begin(), der er en del af Arduino-frameworket og ikke kan kompileres i Microchip Studio. Vi blev derfor nødt til at skifte til Arduino IDE 2, som har dette framework indbygget og dermed kunne kompilere og overføre koden uden problemer.

Da vi skulle få Arduino og Raspberry Pi til at kommunikere, opdagede vi at vi ikke på forhånd havde taget stilling til hvordan dette skulle foregå i praksis. Oprindeligt havde vi planlagt at bruge en enkelt Arduino til både at håndterer sensorer og SOMO-modulet, men vi oplevede at den serielle port ikke kunne håndtere tovejskommunikation stabilt. Arduino'en skulle både modtage kommandoer fra Raspberry Pi'en og sende sensordata tilbage. Vi løste det ved at opdele ansvaret på to separate Arduinoer, hvor sensor-Arduino'en udelukkede sender målinger til Raspberry Pi'en, og SOMO-Arduino'en udelukkende modtager lydkommandoer fra den. Hver Arduino fik sit eget program med en setup-funktion der initialiserede den serielle forbindelse. På Raspberry Pi-siden tilføjede vi to nye klasser i C++ koden der håndterer henholdsvis modtagelse af sensordata og afsendelse af lydkommandoer. Det gav os en forståelse for at kommunikation mellem forskellige hardwareplatforme kræver en eksplicit protokol, og at det er noget med bør tage stilling til tidligt i udviklingsforløbet.

**Uforudsete problemer med seriel kommunikation og bufferophobning**

Under vores første accepttest opdagede vi et uforudset problem med bufferophobning i den serielle kommunikation. Vi oplevede et delay på op til 2 minutter fra en session blev startet, til live-data begyndte at vise de korrekte værdier. Vi forsøgte indledningsvist at afhjælpe problemet hardwaremæssigt ved at binde jordledninger omkring signalledningen for at reducere støj, men det havde ingen effekt, hvilket ledte os videre til at undersøge vores software.

Problemet viste sig at skyldes at Arduinoen løbende sender data uafhængigt af om der er en aktiv session. I perioden mellem to sessioner fortsatte målinger derfor med at fylde serial-portens buffer op, og når en ny session startede, skulle systemet læse hele den ophobede buffer igennem inden det nåede frem til de nye målinger. Vi havde ikke fanget dette problem tidligere, da vi under udviklingen primært havde kørt korte testsessioner hvor vi nulstillede Arduinoen mellem hver test, så bufferen aldrig nåede at bygge sig op.

Løsningen var at implementere en drain() metode i SensorReceiver klassen, som tømmer både programmets egen buffer med __fpurge og operativsystemets buffer med tcflush inden hver ny session starter. Efter implementeringen af drain() oplevede vi ikke længere forsinkelser, og live-data blev vist korrekt fra sessionens start.

# 10. Konklusion

Målet med projektet var at udvikle et system til opsamling af data ift. vand- og energiforbrug og synliggøre disse data live for en bruger imens de er i brusebad. Med dette vil brugeren blive mere bevidst om deres ressourceforbrug og derfor have mulighed for at træffe et faktabaseret valg om at ændre deres adfærd i brusebadet ift. badlængde og temperatur. Undervejs i badesessionen ville system yderligere påminde brugeren om deres forbrug ved et auditivt signal pr forbrugt litermængde vand. Systemet skulle ligeledes understøtte en rangliste over husstandens brugere, som kunne motivere og nudge den enkelte til at reflektere over deres eget ressourceforbrug.

Indledningsvist var målsætningen at systemet skulle implementeres på et eksisterende brusearmatur, men undervejs i processen opdagede vi at en sådan implementering ville tage fokus fra hovedformålet med systemet, netop at måle data, og synliggøre det for brugeren.

I projektet har vi lært at gå fra at analysere og udvælge en given problemstilling fra den virkelige verden, til at beskrive et system igennem funktionelle og ikke-funktionelle krav og use cases, der kunne løse denne problemstilling. Ud fra disse krav og use cases har vi modelleret systemet gennem SysML og UML-diagrammer, for at få en indsigt i, hvorledes systemet kunne kodes og realiseres.

Undervejs i kodeprocessen har vi oplevet at det system som vi har beskrevet og modelleret via diagrammer, havde nogle uforudsete elementer, både i form af manglede bindeledsklasser, samt metoder der sikrede en god seriel kommunikation uden bufferophobning af læst data.

Projektet har resulteret i en fungerende prototype af systemet, så selvom det ikke blev implementeret på et eksisterende brusearmatur, illustrerer prototypen de potentialer der ligger i systemet.

# 11. Fremtidigt arbejde

**Videreudvikling af systemet:**

Med en succesfuld prototype test, er der dannet grundlag for at projektet vil kunne videreudvikles og tage det næste skridt. Her vil opgaven være at undersøge hvorledes man kunne lave en løsning, som vil kunne integreres på et eksisterende brusearmatursystem i et privat hjem, som jo var vores indledende ambition.

I denne fase vil det være essentielt at have fokus på at skabe en sikker løsning hvor vand og elektricitet adskilles mest muligt. Her vil lange ledninger og fritlagte Arduinoer, Raspberry Pi samt en ikke vandtæt touchskærm ikke være en god løsning. Udstyret vil højest sandsynligt tage skade af langvarig eksponering til fugtigt klima med vanddamp, hvad der jo ofte er på et badeværelse efter et brusebad eller lignende.

**Trådløse løsninger og brugeroplevelse:**

Et oplagt område at undersøge og teste, vil være trådløse sensor løsninger, samt en trådløs kobling mellem skærm og Raspberry Pi / anden mini computer, eller alternativt at lave en app til telefonen, som vil kunne koble sig op på husstandens sensorer.

Her vil det være relevant at undersøge forskellige typer af trådløse sensorløsninger [11]. Dette kunne f.eks. være temperatursensorer, som kan monteres udenpå eksisterende vandrør uden større indgreb, eller trådløse flowmålere som kan sende data direkte til systemet. Derudover vil det være muligt at undersøge løsninger hvor sensorer, skærm og backend kommunikerer trådløst via f.eks. wi-fi. På den måde vil installationen kunne gøres mere fleksibel og samtidig reducere mængden af ledninger i badeværelset.

Mange af disse løsninger vil dog fortsat kunne være teknisk krævende at installere for almindelige brugere, særligt hvis systemet kræver ændringer på eksisterende vandinstallationer eller ekstra hardware for at fungere stabilt. Derfor vil det i den videre udvikling af systemet være vigtigt at have fokus på brugervenlighed, simpel opsætning og en løsning som er nem at integrere i private hjem.

Uanset hvilken af disse løsninger der vælges, vil det være vigtigt at holde fast i den indledende tanke: Brugeren skal kunne følge med live ift. deres vandforbrug. Dette er essentielt fordi ligesom når man står og tanker benzin og ser liter mængden løbe op og derved også regning, så bliver brugeren bevidst om deres forbrug, og kan være med til at igangsætte refleksion og derved måske ændre deres forbrugsmønster. På den måde kan man argumentere for at på trods af det vil være utrolig smart med en app, så af person sikkerhedsmæssige årsager så vil en fast vandtæt skærm uden kamera være en god løsning.




*Figur 45: Tavle ved sø, med nudging fakta til brugere.*




*Figur 46: Benzin stander, live data*

**Installation og målgruppe:**

Ud over de ovennævnte fokuspunkter vil man også skulle overveje hvorledes systemet skal installeres i en privat husstand og træffe valg ift. om brugeren selv skal kunne installere systemet, eller om det skal være påkrævet at det bliver installeret af en professionel montør, f.eks. en VVS'er. Man kan argumentere at hvis systemet ikke krævede en professionel montør til at installere systemet, vil man potentielt kunne nå ud til en større målgruppe, da en dyr VVS-regning potentielt kunne være afskrækkende for nogle bruger. Sådan et valg sætter dog ekstra krav til brugervenlighed og installation af systemet.

# Bilagsoversigt

**Bilag A**: Kravspecifikation

**Bilag B:** Accepttestspecifikation

**Bilag C:** Systemarkitektur

**Bilag D:** Design og implementering

**Bilag E:** Gennemført Accepttest

**Bilag F:** Datablade

**Bilag G:** Vejledninger til semesterprojektet

**Bilag H:** Procesbeskrivelse

**Bilag I:** Proces dokumenter (Referater, samarbejdskontrakt samt mødeindkaldelser)

**Bilag J:** Kodefiler

**Bilag K:** GAI-deklaration

# Referencer


[1] Wikipedia, »MoSCoW method,« Wikipedia, The Free Encyclopedia, [Online]. Available: https://en.wikipedia.org/wiki/MoSCoW_method. [Senest hentet eller vist den 2026].

[2] K. Bjerge, »Vejledning til udviklingsprocessen for projekt 2,« Lokal kopi forefindes i bilag G, 2014.

[3] M. Fowler, UML Distilled: Third Edition, 2003.

[4] L. Delligatti, SysML Distilled: A Brief Guide to the Systems Modeling Language, 2013.

[5] I2ISE, »System Application Models,« Lokal kopi forefindes i bilag G.

[6] YF-S201 datasheet, »YF-S201,« Lokal kopi forefindes i bilag F.

[7] Maxim integrated, »DS18B20,« Lokal kopi forefindes i bilag F.

[8] Danwa, »Danwa,« [Online]. Available: https://www.danva.dk/media/11419/vand-i-tal-2025-k10-final2.pdf. [Senest hentet eller vist den 2026].

[9] D. statistik, »Danmarks statistik,« [Online]. Available: https://www.dst.dk/en/Statistik/emner/miljoe-og-energi/energiforbrug-og-energipriser/energipriser. [Senest hentet eller vist den 2026].

[10] Stackoverflow, »Stackoverflow,« [Online]. Available: https://stackoverflow.com/questions/997946/how-can-i-get-current-time-and-date-in-c. [Senest hentet eller vist den 2026].

[11] tektelic, »guide-to-lorawan-sensors,« tektelic, [Online]. Available: https://tektelic.com/expertise/guide-to-lorawan-sensors/. [Senest hentet eller vist den 2026].